

Architect

Architecture, HLD & LLD

2.0 / Build an application

3.0 / Connect the company

4.0 / Own the infrastructure

Darsh Dave

27 September 2026

Working prototype + proposed target design

4 labelled GPT-generated diagrams · 16 product areas

109 feature packages · 422 inherited interactions · 228 handoffs

Submission companion: ARCHITECTURE.md contains the full per-feature edition scopes, actors, journeys and recovery paths.

Document map

Reading guide and scope

Product roadmap and architectural boundaries

Current system: what actually runs

HLD: Architect 2.0

HLD: Architect 3.0

HLD: Architect 4.0

Architecture decisions and trade-offs

LLD: project and artifact model

LLD: agent harness and execution

LLD: context and memory

LLD: API, events and concurrency

LLD: sandbox, tools and GitHub

LLD: models, costs, quality and release

LLD: company handoffs and private operations

Failure cases, verification and rollout gates

Source references and inspection basis

Appendix A: product and feature architecture coverage

Appendix B: inherited interaction traceability

Appendix C: cross-product handoff register

Appendix D: shared experience and later utilities

Reading guide and scope

This document presents the architecture of the working prototype and a proposed technical design for the product roadmap. It covers Architect 2.0, 3.0 and 4.0. HLD means high-level design: system boundaries, responsibilities and data movement. LLD means low-level design: records, interfaces, state transitions and recovery rules.

Implementation snapshot: repository commit 72c923c. The architecture submission is a documentation addition, not an implementation of the proposed runtime. The website Architecture and HLD & LLD tabs are intentionally deferred until approval.

Status vocabulary: Current = supported by the inspected code; Prototype = a saved configuration or interactive demonstration, not an external action; Proposed = the target design described here. A feature appearing in the appendix establishes architectural traceability, not proof of production completion.

There are 109 proposed feature packages: 40 enhancements and 69 new packages. Architect 2.0 introduces 65, 3.0 introduces 42 more (107 cumulative), and 4.0 introduces 2 more (109 cumulative). The 422 inherited interaction records are a separate measurement and must not be added to 109. Appendices preserve all package IDs, all inherited interaction IDs and all 228 documented cross-product handoffs.

Product roadmap and architectural boundaries

Edition	User need	Architectural increment	Current delivery
2.0	Individuals and developers build, inspect and own an application	Project context; developer workspace; agent and model configuration; quality and release workflows	React application, authenticated persistence, bounded AI generation and interactive workflows
3.0	A company carries decisions from feedback through delivery	Company membership; role views; shared artifacts; bot teams; business connectors; approval and metric definitions	Shared company data/access code plus role and business workflow prototypes
4.0	A company controls where its platform and execution run	Private control plane; customer identity, network, secrets, worker placement and lifecycle	Hosted configuration prototype and container scaffolding; no customer-cloud installation verified

All three hosted editions currently share one codebase and one Supabase account/data foundation, with three frontend deployments selected by VITE_EDITION. Separate origins do not mean an automatically shared browser session: each origin authenticates and retrieves the same authorized data. The assignment is a fourth, static deployment.

A real private 4.0 installation is a different trust boundary: its database and identity must be customer-controlled, not silently connected to the public demo database. An explicit, audited migration can transfer permitted projects. Shared-schema compatibility is the continuity mechanism, not unrestricted cross-instance data sharing.

Current system: what actually runs

The browser loads a React 19 / TypeScript / Vite application from Vercel. Supabase handles Google/email authentication and Postgres persistence. The client uses Supabase HTTP APIs; the server AI endpoints are Node functions. OpenRouter is called only from the server. Provider keys are not shipped to the browser. GoDaddy supplies DNS records pointing the custom domains to Vercel; Google Cloud configures the OAuth client, not a separate app runtime.

Current request sequence: user signs in; the API validates the bearer token with Supabase; validates input and model constraints; fetches the free-model catalog; reserves usage through a database function; calls OpenRouter; returns text, model, usage and request ID. The app stores accepted output and project state. Generation is synchronous, not a durable background job, and there is no autonomous multi-agent executor in this path.

Current responsibility	Code evidence	Boundary
Editions and views	apps/platform/src/lib/edition.ts; App.tsx	One application with edition-specific navigation
Personal projects	src/lib/cloud.ts; supabase/schema.sql	Owner-scoped JSON workspace and optimistic revision checks
Company membership	supabase/company-schema.sql; CompanyAccess.tsx	Company roster, invitation and workspace routines; broader granular policy is proposed
Model requests	api/generate.js; api/_lib/core.mjs	Modes app, plan, chat, support; authenticated requests; free models only
Project context	src/lib/builderState.ts	Brief, PRD/TRD, HTML, attachments and workflow serialized within a bounded context
Preview isolation	src/lib/preview.ts; Studio.tsx	Browser iframe with restrictive CSP; not a server sandbox
Agent playground	Workbench.tsx	Configured role/goal, project brief and six recent messages; prompt-only simulation
Home support	SupportChat.tsx; support-knowledge.json	Signed-in AI guide; demo saved responses; no account actions or human inbox
Infrastructure	InfrastructureStudio.tsx; PrivateLifecycle.tsx; Dockerfile	Configuration and lifecycle prototypes; not provisioned cloud resources

Current personal workspace rows are capped at 1.2 MB JSON and company workspace rows at 2.4 MB. These are prototype storage choices, not the target normalized schema. Current quota SQL allows 10 requests per account per UTC day, 40 globally, and a 15-second interval. A provider failure can consume a reserved attempt. These limits describe this code snapshot, not a commercial pricing promise.

The current builder bounds context at about 39,000 characters; the API accepts up to 40,000 context characters and 12,000 prompt characters. Truncation preserves a smaller serialized excerpt; it is not semantic retrieval or summarization. Generated previews are self-contained HTML with inline scripts and styles. CSP blocks network requests; sandbox isolation does not make this a full framework runtime. Large code generation and repository execution require the proposed execution plane.

Known implementation gaps: real GitHub OAuth/clone/sync/PR execution; a repository-backed IDE; durable agent jobs; tool execution through Lyrz; long-term memory retrieval; external connector actions; provisioning; real release orchestration; enterprise SSO and granular department policies. These are not claimed live. The container scaffold also needs packaging validation: `core.mjs` imports the support knowledge JSON from `src/lib`, while the runtime image currently copies `api` but not that source path. An installable 4.0 release must fix and smoke-test that dependency before distribution.

HLD: Architect 2.0

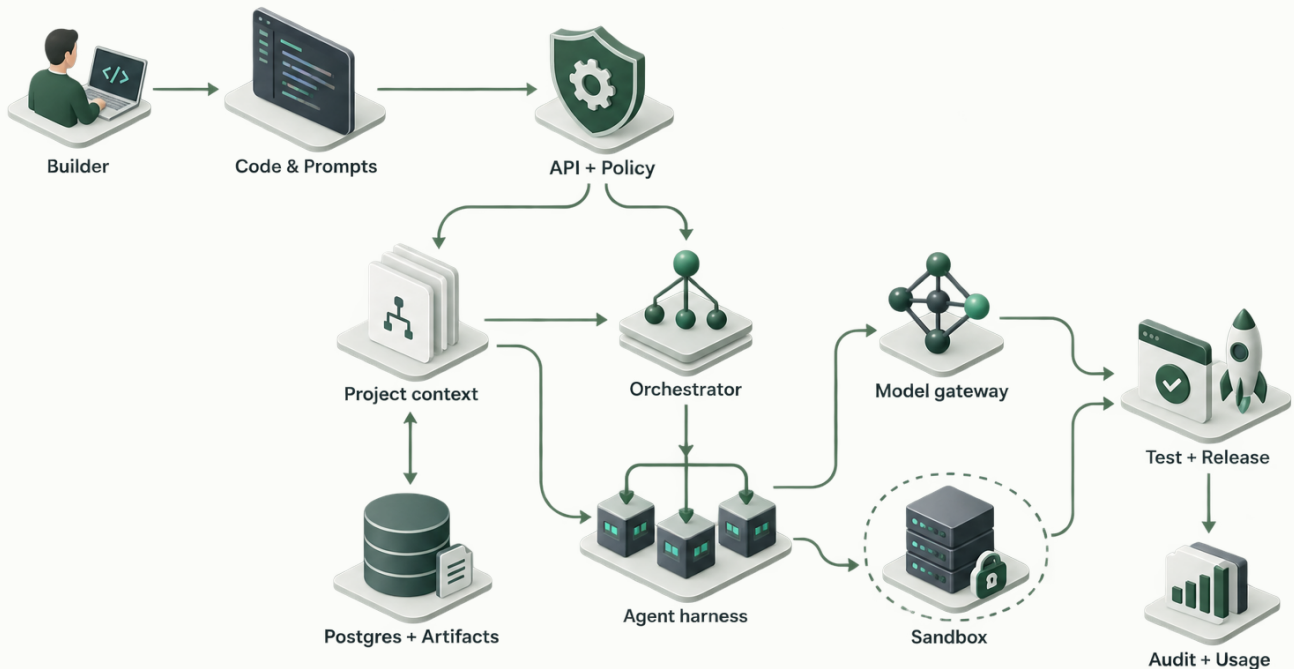


Figure 1. Proposed developer platform. The visual illustrates responsibility flow, not a literal deployment topology. All worker, repository and durable-memory services in this target are proposed.

Start with a modular TypeScript API and Postgres, keeping product modules together until load or isolation justifies a service split. The browser provides guided and developer modes, a file tree, editor, chat, visual preview and project switcher. Monaco is the proposed editor; it edits a checked-out workspace through authorized file operations. It is not currently integrated.

The control plane owns identity, memberships, project artifacts, run creation, policies, budget reservations and approvals. A durable workflow service coordinates planner, builder, reviewer, tester and release activities. A separate execution plane runs untrusted repository code. It never executes inside the API process or the control-plane database network.

A project begins from a prompt or an approved GitHub App installation. Import resolves repository permissions and a commit SHA, inventories files and frameworks in an isolated environment, and creates an initial context snapshot. Build output becomes a changeset against that exact base revision. Agents cannot write directly to protected branches. Diff review, automated checks and approval precede a provider-confirmed release.

Proposed technologies: React/TypeScript and Monaco for the browser; a TypeScript modular API; Supabase/Postgres for relational state; S3-compatible object storage for large artifacts; Temporal for long-running workflows; a Lyzr runtime adapter for supported agents; OpenRouter through a policy-controlled gateway; microVM execution via a sandbox provider. Framework-specific agents run behind the same adapter contract rather than being advertised as universally compatible without validation.

HLD: Architect 3.0

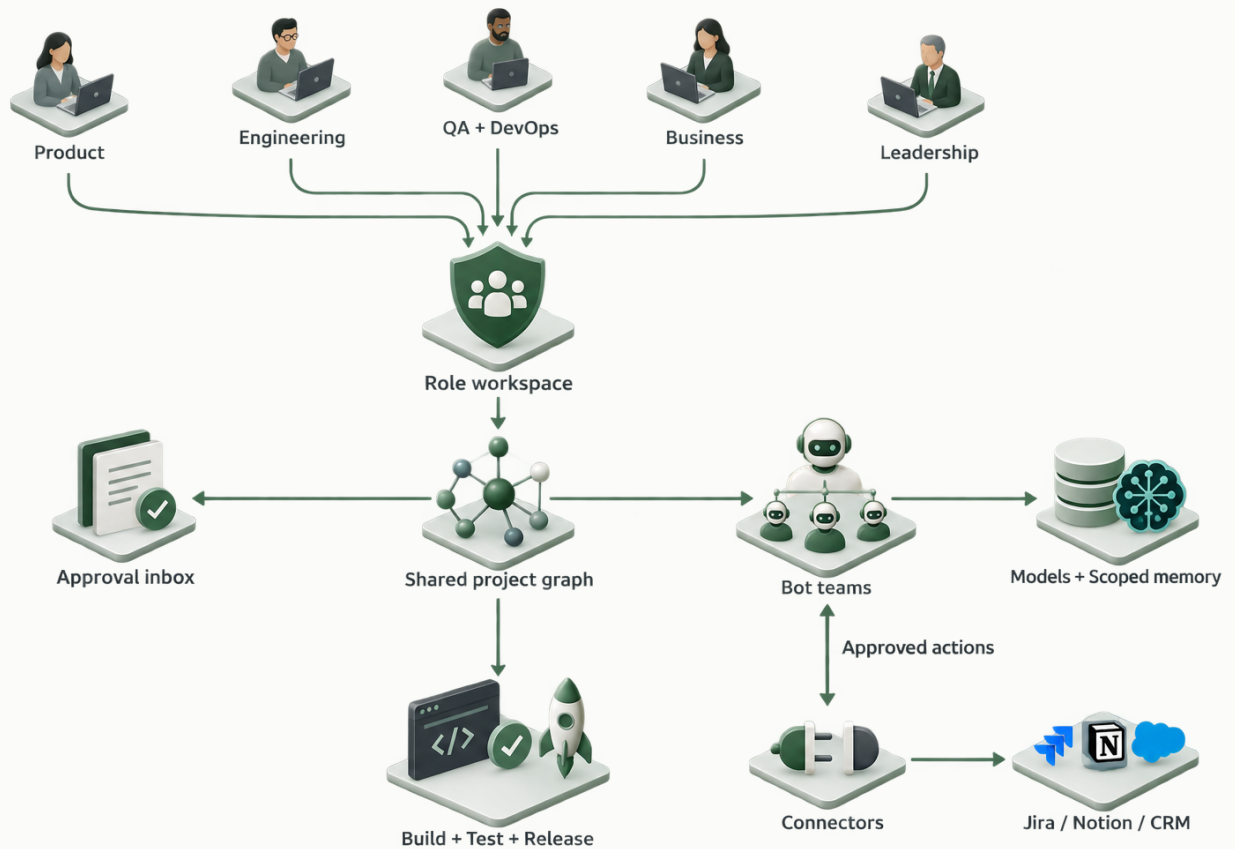


Figure 2. Proposed company workspace. Role-specific views converge on shared authorized objects, approval steps and delivery evidence. Business connectors and bot execution are target capabilities.

3.0 retains the 2.0 build engine. It adds company-scoped records for customers, feedback, requirements, delivery tasks, campaigns, financial review, people operations, metrics and decisions. These are linked by stable IDs in Postgres; a graph database is not required initially. Product, engineering, QA, DevOps and business stakeholders see the same underlying objects through different permitted views.

An example handoff is feedback to requirement to changeset to test run to release to outcome metric. Navigation carries project ID, artifact revision, permitted selection and return location. Authorization is rechecked at the destination. A saved return route never overrides access control. UI roles are preferences; database policies and server action checks determine permissions.

The bot workroom defines a lead bot and specialists. The lead proposes tasks; the orchestrator authorizes bounded child runs and the tool broker executes approved actions. Specialists do not gain access by being created by a manager bot. A scheduled report rechecks the owner's membership, data scope, recipient policy and budget at dispatch time.

Connectors to Jira, Notion, CRM and data systems use a common adapter with explicit field ownership, provider IDs, sync cursors and deduplication. The external system remains authoritative for its owned fields. Webhook events are verified and queued; conflicting updates become reviewable conflicts instead of last-write-wins data loss. No outbound campaign, financial action or HR decision is executed merely because an agent suggests it.

HLD: Architect 4.0

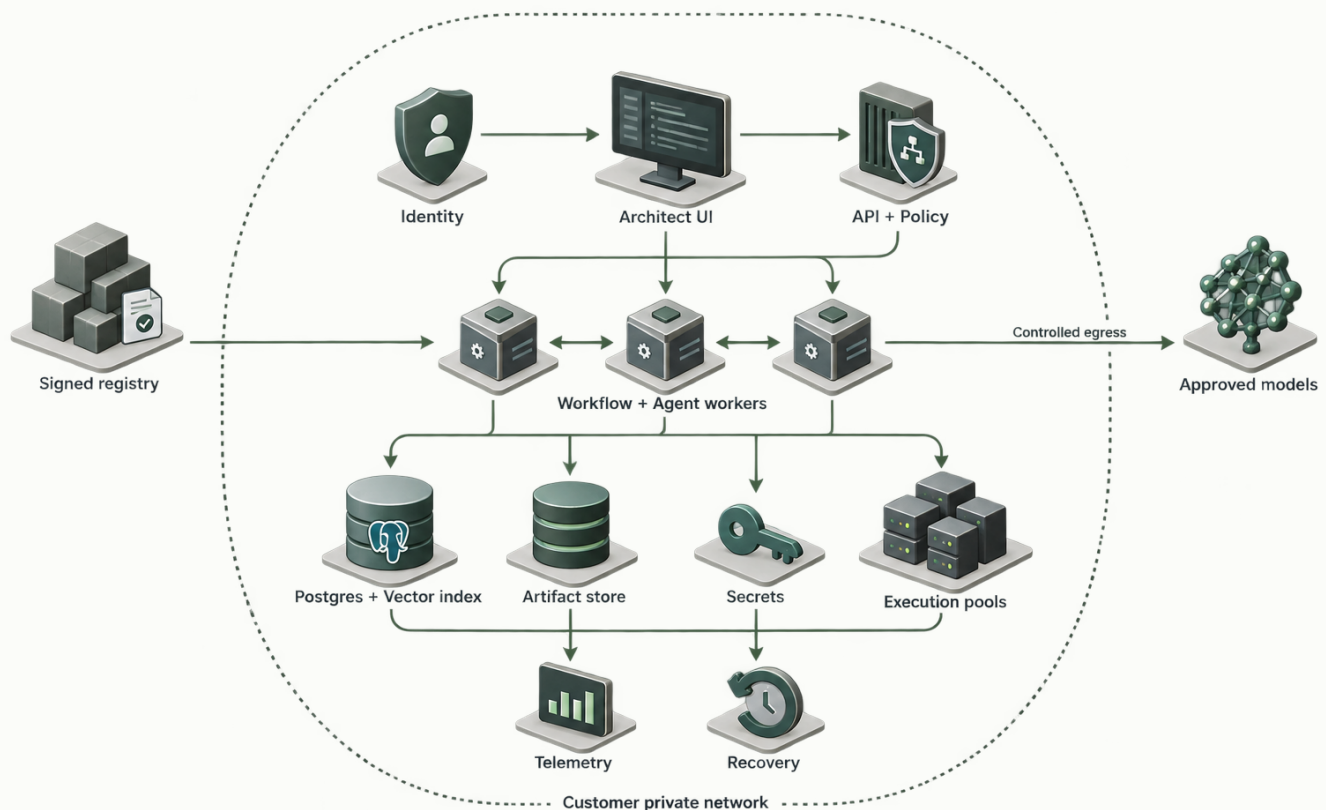


Figure 3. Proposed customer-controlled deployment. Identity, secrets, storage, workers and telemetry sit within the customer's boundary. Registry updates and approved model requests are explicitly controlled connections.

4.0 packages the control plane, workflow workers and connector gateway as versioned OCI images. Docker Compose is the proposed evaluation installation; Helm/Kubernetes is the proposed managed production form. The same application contracts support AWS, Azure and GCP through deployment adapters, not three separate product implementations. Provider support must be tested against a declared compatibility matrix.

The company connects its identity provider through OIDC, with SAML through an approved identity broker where required. Installation bootstrap appoints an initial owner once, then disables the bootstrap route. There is one company sign-in entry; role assignment is administrative. SCIM lifecycle integration is proposed, with revoked users removed from active sessions, schedules and connector delegation.

Each environment references a signed runtime profile: network, region, execution pool, limits, storage, egress policy and secret references. Customer workload identity replaces copied cloud administrator keys. Execution pools are separated from the control plane; per-run sandboxes are disposable and receive narrowly scoped credentials. Private model endpoints are allowlisted; disabling public model egress must fail closed, not fall back to a public provider.

Upgrades verify signatures, back up data, check schema compatibility, migrate through a controlled job, then run health checks. App rollback alone cannot reverse a destructive schema migration; use expand/contract migrations and a tested restore path. Encrypted backups, artifact replication, restore drills and customer-selected retention are required. Proposed initial objectives are a 24-hour recovery-point objective and a 4-hour recovery-time objective, subject to measured restore tests and customer requirements; no SLA is claimed.

Architecture decisions and trade-offs

Decision	Choice and rationale	Trade-off / acceptance gate
One project model	Shared IDs and artifact contracts across editions	Private installations need explicit migration, not implicit synchronization
Modular API first	Postgres transactions and clear module ownership; workers isolated	Split services only for scaling or security boundaries
Durable control flow	Temporal owns retries, waits and approval resumption	Adds operational cost; not installed in the free prototype
Agent runtime adapter	Lyzr preferred where supported; framework adapters normalize runs	Verify cancellation, tools, memory and tenancy semantics per adapter
Model routing	Explicit model or policy-filtered per-task selection	No quality claims without held-out evaluations; no silent paid fallback
Storage	Postgres is authoritative; object storage holds binaries; pgvector is a derived index	More lifecycle management; index deletion must track source deletion
Git authority	GitHub App, short-lived installation access, pinned commits	App permissions and provider installation remain to be built
Untrusted execution	MicroVM boundary, egress policy and per-run credentials	Plain Docker alone is insufficient for hostile multi-tenant code
Delivery evidence	Test results tied to a changeset and environment	A model saying 'passed' is never a test result
Free prototype	Existing free-model guard and bounded synchronous requests	Proposed worker, sandbox and enterprise stack is not guaranteed free

LLD: project and artifact model

Proposed authoritative entities are normalized by tenant and project. IDs below are design contracts, not existing database migrations. Every child reference must enforce matching tenant/project ownership through composite foreign keys or an equivalent database check; checking only an object UUID is insufficient.

Entity	Key fields	Invariant
Tenant / Membership	tenant_id, user_id, role, policy_version, status	Membership is checked server-side; revoked status denies access
Project	id, tenant_id, owner_id, title, revision, lifecycle	Revision is monotonic; writes require expected_revision
ArtifactRevision	id, project_id, kind, version, content_hash, object_ref, source_refs, approval	PRD, TRD, HLD, LLD, design system, code and test evidence are distinct artifact kinds
RepositoryBinding	project_id, installation_id, repo_id, branch, base_sha	Credentials stay in the secret service; provider permission is authoritative
Service	id, project_id, repo_id, path, contract_ref, dependencies	Supports monorepo paths and several repositories without losing commit identity
AgentDefinition	id, project_id, version, role, instructions_ref, model_policy, tool_allowlist, memory_scope	A run pins a definition version; edits do not alter an in-flight run
Run / Step	id, project_id, revision, actor, policy_version, budget, status, parent_run_id	Bounded state transitions; one durable correlation ID
ChangeSet	id, base_sha, file_manifest, diff_ref, author_run_id	Apply only to expected base; conflict requires rebase/review
TestRun / Release	changeset_id, environment, evidence_refs, result, provider_receipt	Readiness depends on real checks and provider acknowledgements
Approval	id, action_hash, resource_revision, reviewer, expires_at, decision	Approval covers a specific action and revision; cannot approve a modified action
ToolInvocation	run_id, tool_id, args_hash, idempotency_key, receipt, status	Retry only with the same logical action key; reconcile unknown outcomes
UsageLedger	run_id, agent_id, model, provider, module, units, cost, currency, source	Actual provider cost, estimated cost and unknown cost remain distinct
ConnectorBinding	tenant_id, external_id, scopes, secret_ref, owner, status	Tokens never enter prompt context; revoke promptly

PRD to TRD to HLD to LLD is an artifact dependency chain. A PRD revision marks dependent designs as potentially stale. An agent proposes updates as new revisions with a diff. The user may skip manual document authoring; the planner creates drafts automatically and makes assumptions visible. An approved plan does not automatically approve deployment.

For example, requirement R-12 references PRD revision 4, a service contract revision 2, changeset C-18 and test run T-21. When C-18 changes, T-21 stays as historical evidence and does not certify the new change. This is how the system avoids stale 'green' status.

LLD: agent harness and execution

An agent harness is the controlled runtime around the model: context loading, model calls, tool validation, output checks, checkpointing and budgets. The model proposes work; the harness and policy service decide which actions may execute.

The proposed adapter contract accepts `run_id`, `tenant_id`, `project_id`, `agent_definition_version`, `context_manifest`, `model_policy`, `tool_scopes` and `limits`. It emits `step_started`, `model_result`, `tool_requested`, `artifact_proposed`, `approval_required`, `failed` and `completed` events. Outputs are typed artifacts with source references. Providers' raw tool calls are normalized before policy evaluation. Lyzr would sit behind this contract; Architect still owns tenant policy, approval records and durable workflow state.

Run lifecycle: `queued` -> `preparing` -> `running` -> `awaiting_approval` or `validating` -> `completed`. Failure exits are `failed`, `cancelled` and `timed_out`. An approved wait resumes only after checking current membership, expiry and the action hash. Cancellation propagates to child runs, stops new tool dispatch and tears down the sandbox; already completed provider actions must be reconciled, not reported as `undone`.

Initial proposed defaults: five concurrent specialists per parent, maximum delegation depth three, twenty tool calls per step, ten-minute sandbox execution, and a per-run token/cost cap. They are server-enforced configuration, not prompt suggestions or measured product limits. Dynamic specialists inherit the intersection of parent permissions, project policy and approved template scopes. A lead bot cannot spawn around an approval denial.

Each tool call uses a schema validator and an allowlisted tool identity. Read, write and destructive operations have separate scopes. Tools return evidence and structured errors. A release activity may need a human approval; a code-reading activity generally does not. Replaying a workflow reuses recorded model decisions where possible and never reruns external side effects without checking the invocation ledger.

Testing uses specialists with separate evidence responsibilities: unit/functional, UI/accessibility, integration/contract, security and agent-behavior evaluation. A reviewer aggregates their results against acceptance criteria. Specialists are selected based on the app manifest; a static page should not acquire a fake database test. UI repair is bounded by attempt count and must rerun affected checks. The model cannot change the acceptance criteria silently to make a test pass.

LLD: context and memory

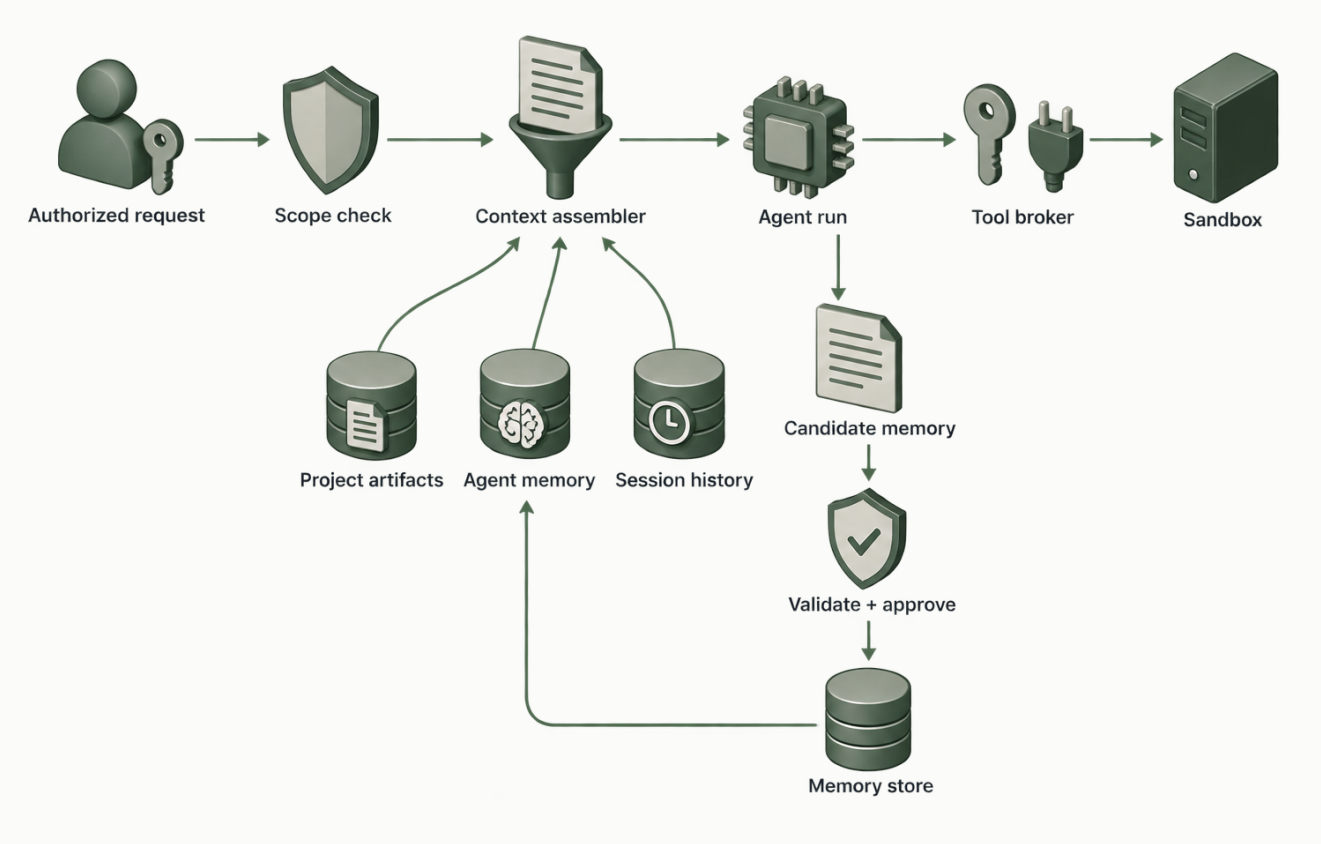


Figure 4. Proposed context and memory flow. Component labels identify the request path, scoped context inputs, controlled tool execution and validated memory-write loop.

Project context and memory are separate. Project artifacts are explicit, versioned sources of truth; memory is a permitted retrieval aid. A conversation summary cannot override an approved requirement or a repository commit.

Scope	Contents	Access and lifecycle
Session	Recent messages, task state, summary and user decisions	Session principal plus assigned run; bounded retention, no implicit company sharing
Agent	Agent-specific lessons and working notes	tenant + project + agent; other agents need an approved promotion or explicit access
Project	Approved requirements, architecture, service map, code references and decisions	Project membership and resource policy; revisioned source references
Company	Approved reusable policies, templates and metric definitions	Group-aware access, explicit publishing; private HR/finance records excluded by default

Scope	Contents	Access and lifecycle
Execution	Tool receipts, test evidence, traces and checkpoints	Run-scoped audit access and retention; secrets redacted

Proposed MemoryRecord fields: memory_id, tenant_id, project_id, agent_id (nullable only for shared scopes), scope, source_artifact_id, source_revision, text_ref, content_hash, embedding_ref, sensitivity, provenance, created_at, expires_at, approval_status and tombstoned_at. A unique constraint on scope plus source revision plus content hash prevents duplicate ingestion. Embedding-model version is stored so incompatible vector indexes are never mixed.

Retrieval sequence: authenticate -> resolve permitted scope -> filter by tenant/project/agent, current ACL, approval and expiry -> fetch candidates -> rank -> verify source revision -> assemble a token-budgeted context manifest. Filtering happens before data can reach the model, including vector retrieval. Recheck ACL and source freshness before a tool uses retrieved data. Sensitive company records never enter a broad shared index merely because their embeddings are hard to read.

The context manifest pins instruction version, model policy, artifact IDs/revisions, retrieved memory IDs, source hashes and truncation notices. Allocate the budget to the user's task and approved requirements first, then relevant code and tools, then retrieved memory, then recent conversation. Use model tokenizer limits rather than a character count. Oversized context is summarized with citations and disclosed omissions; source artifacts remain retrievable.

Memory write sequence: agent proposes a candidate -> validate schema and provenance -> reject secrets and unsupported claims -> apply scope/retention policy -> require approval for shared promotion -> persist -> update derived indexes. Contradictions generate a review item; they do not silently overwrite approved facts. Deleting a source tombstones associated memories, removes derived index entries and invalidates caches. Retention for backups must be disclosed separately; legal holds are explicit policy exceptions.

LLD: API, events and concurrency

The following endpoints are proposed, except the existing `/api/generate`, `/api/models` and `/api/health`. Endpoints require authenticated membership and resource checks independent of UI visibility.

Proposed interface	Request / result	Concurrency and failure rule
POST <code>/v1/projects/{id}/runs</code>	intent, base_revision, context_refs, model_policy -> 202 run_id	Idempotency key scoped to tenant and action; return existing run on duplicate
GET <code>/v1/runs/{id}/events</code>	last_event_id -> ordered event stream	Reconnect resumes from cursor; unauthorized clients receive no events
POST <code>/v1/runs/{id}/approvals</code>	action_hash, expected_revision, decision	409 for stale revisions; expired approvals cannot resume work
POST <code>/v1/projects/{id}/changesets</code>	base_sha, manifest, diff_ref	Validate paths and ownership; never trust agent-supplied file paths blindly
POST <code>/v1/memory/query</code>	scope, query, source revision constraints	Scope derived from authenticated principal; client cannot widen it
POST <code>/v1/connectors/{id}/actions</code>	action, args, approval_ref	Recheck provider scopes and use idempotency/reconciliation
POST <code>/v1/releases</code>	changeset, environment, evidence, approval_ref	Publish only a reviewed artifact; provider receipt required
POST <code>/v1/installations/{id}/upgrades</code>	signed_version, backup_ref, maintenance_window	Customer admin only; restore plan and compatibility checks required

Event envelope: event_id, schema_version, type, tenant_id, project_id, aggregate_id, aggregate_revision, run_id, actor_id, occurred_at and payload_ref. The transactional outbox writes a domain change and its event together. Consumers deduplicate event_id and track aggregate revision. Delivery is at least once; the design does not promise exactly-once external execution.

A document edit uses expected_revision; stale edits produce a conflict rather than overwriting another user's work. A sandbox filesystem is isolated per changeset; competing changes are merged with a visible diff. Multi-repository builds pin all service SHAs in a manifest and record contract versions. No partial service release is treated as a coordinated success.

LLD: sandbox, tools and GitHub

Current preview protection is a browser iframe plus CSP, not a secure arbitrary-code compute service. The target uses disposable microVMs or equivalent strong isolation. Plain containers may package trusted workers, but are not the sole security boundary for untrusted multi-tenant code.

Sandbox creation receives an immutable base image digest, project snapshot, resource limits, allowed network destinations, expiry and short-lived tool credentials. It runs without host mounts or control-plane credentials. Block cloud metadata endpoints, private network routes and unapproved egress. Dependencies are downloaded through a policy-controlled path with lockfiles and supply-chain checks. Logs are redacted before persistence. File artifacts pass size/path/type checks before export. The worker records exit status and evidence, then destroys the environment on completion, cancellation or TTL expiry.

GitHub uses a proposed GitHub App installation, selected repositories and minimal scoped permissions. The backend exchanges installation credentials; no personal access token is pasted into prompts. Clone/read uses a pinned SHA, writes go to an isolated branch, PR creation carries the changeset ID, and protected-branch rules remain authoritative. Signed webhook payloads are verified, replay-protected and reconciled. Disconnecting an installation cancels pending writes and removes credentials.

The tool broker also governs MCP servers and plugins. Store an allowlisted server identity, verified schema, allowed resources, delegated principal and secret reference. Treat tool descriptions/results and imported repository text as untrusted input. Prompt instructions cannot expand scopes, reveal secrets or skip approvals. External content may inform a decision; it cannot authorize one.

LLD: models, costs, quality and release

The proposed routing order is: explicit user choice -> policy eligibility (privacy, region, capability, price, budget) -> task-specific evaluation score -> latency/cost tie-breaker. Single-model mode pins one eligible model; per-agent mode pins eligible models by specialist; automatic mode records the selected model and routing reason for each step. A user choice that violates company policy returns an explanation, not an invisible substitution.

BYOK credentials live in the customer's secret store and are referenced by ID. Open-weight endpoints require a compatible inference adapter and evaluation; 'open source' does not imply free compute. The current prototype only uses its configured server OpenRouter key and free models. Proposed fallback routes require explicit price/privacy bounds and record the fallback. Current code has no paid fallback.

Cost records separate provider model charges, Architect charges, module attribution, tool/API charges and cloud resource charges. Attribute run -> agent -> model -> API call -> project -> tenant. Provider billing exports and cloud tags supply actuals; missing attribution goes into an unallocated bucket. EC2/S3 or equivalent recommendations are estimates with assumptions and require approval before infrastructure changes. A configured budget is not a real payment or invoice.

A release requires evidence tied to the exact artifact: unit tests, UI/accessibility checks, service contracts, agent evaluations and applicable security checks. Proposed tools include Vitest for unit tests, Playwright for browser checks, axe-core for accessibility and an image/container scanner appropriate to the stack. CI runner results are stored as evidence; failed checks stay visible. Staging previews use isolated test data, never copied production secrets.

Deployment adapters normalize plan, dry_run, deploy, status and rollback. Capture provider receipts and verify an application-level health check after deployment. Canary rollout needs monitored thresholds and an explicit rollback target. Schema changes use expand/contract migrations. Provider timeout leaves status unknown until reconciled; it must not create a second deployment automatically.

LLD: company handoffs and private operations

All 228 documented handoffs share a ContextEnvelope: tenant_id, project_id, source_object_id, source_revision, destination_type, permitted_selection, navigation_state and correlation_id. The destination resolves an existing linked object or previews creating a new one. A back action restores filters, scroll, draft and tab without replaying an external action. The envelope carries references, not a full copy of sensitive records.

Company administrator flows: create company -> verify owner -> configure departments and roles -> invite a named email or provision through approved SSO -> accept/resolve identity -> show permitted role home. Email domain alone never grants membership. Personal-account preference changes do not grant company access. Current company SQL implements basic owner/admin/member/viewer distinctions; target department/field/action policies need additional server enforcement.

Finance, HR, customer and sales records have separate policy scopes. Human reviewers own financial commitments and employment decisions. Team productivity views use explicit goals, workload and acknowledged context, not covert monitoring or an automated employee ranking. Agent teams may propose a response or report, but the person accountable for publication or a sensitive decision remains identifiable.

For 4.0, separate deployment profiles for development, staging and production contain cloud region, subnet, worker identity, egress, data retention and budget. Provisioning plans show resources and estimated cost before applying. The installer checks prerequisites, identity, storage, DNS/TLS, network reachability and signing keys. No hidden paid cloud resources are created by the prototype. The design requires a customer-approved bill of materials before a real installation.

Failure cases, verification and rollout gates

Failure	Required behavior	Verification gate
Provider 429 / timeout	Preserve prompt and run; bounded retry; no silent model/cost change	Inject failures and verify budgets and retry ceilings
Worker crash	Resume from recorded checkpoint; reconcile prior tool receipts	Terminate worker after tool success before acknowledgement
Expired session / revoked role	Stop new tool calls; deny retrieval and reconnect	Revoke access during a run and test API plus event stream
Cross-tenant memory request	Reject before retrieval/model exposure	Adversarial ID substitution and vector filter tests
Malicious repository / tool output	Treat as data; block secret requests and arbitrary tools	Prompt-injection, path traversal, SSRF and escape test suites
Concurrent edits	Preserve both changes; explicit conflict resolution	Parallel saves against same revision
Approval becomes stale	Invalidate by action hash or resource revision	Modify planned action after approval and confirm denial
Duplicate webhook / schedule	Deduplicate and reconcile; one logical external action	Replay signed event and simulate acknowledgement loss
Broken deployment	Show provider state; halt rollout; use verified rollback target	Failure drills with app and schema version mismatch
Deleted memory source	Tombstone, remove derived index and invalidate caches	Query after deletion across all retrieval routes
Private upgrade failure	Stop rollout, retain evidence, execute compatible recovery	Offline backup restore and failed-migration rehearsal

Verification in this documentation task consists of checking the implementation snapshot, matching canonical scope IDs and rendering the submission files. It is not a new live-provider or private-cloud acceptance test. Existing browser evidence and automated tests remain in docs/evidence and docs/history.

Proposed rollout: first normalize project artifacts and add real GitHub onboarding plus run ledger; then ship isolated execution and verified build/test loops; then durable agent runtime and permissioned memory; then company connectors and bot schedules; then private packaging and restore certification. Each step needs evidence before its status changes from proposed to current. The full enterprise architecture will require hosting resources beyond the present free demonstration.

Source references and inspection basis

Repository sources are authoritative for current behavior; provider documentation informs proposed technology choices. References were checked on 27 September 2026. Provider capabilities do not prove integration in this application.

- Canonical scope: apps/assignment/data.json (109 feature packages, 422 inherited interactions, 228 handoffs). The older platform features.json contains a subset and is not used as the submission scope denominator.
- Runtime: apps/platform/api/_lib/core.mjs; api/generate.js; src/lib/cloud.ts; src/lib/builderState.ts; src/lib/preview.ts; src/lib/model.ts.
- UI and workflows: src/components/Studio.tsx; Workbench.tsx; CompanyStudio.tsx; InfrastructureStudio.tsx; PrivateLifecycle.tsx; src/lib/workflows.ts; company-workflows.ts; company-features.ts.
- Database: apps/platform/supabase/schema.sql and company-schema.sql. These demonstrate schema intent and guards; source inspection alone does not prove every deployed policy was applied.
- Lyzr multi-agent orchestration: documents manager-led and deterministic orchestration; the proposed adapter must validate the selected runtime's actual semantics.
- Temporal workflow execution: basis for the proposed durable workflow choice. External side-effect safety still requires application idempotency and reconciliation.
- Supabase row-level security: basis for database access policies, supplemented here by server authorization and composite ownership checks.
- Vercel Sandbox: candidate isolated execution service; no claim that it is provisioned or free in this project.

Diagrams were generated using GPT image generation, with transparent backgrounds and no embedded page titles. They are explanatory target-architecture views; exact contracts are specified in this text. Generation prompts are saved in diagrams/PROMPTS.md. No diagrams were authored as Python drawings, SVG or XML.

Appendix A: product and feature architecture coverage

Every package below is mapped from the canonical scope. These are target responsibilities; the diagram labels show shared components rather than 109 separate microservices. Figure 1 covers the 2.0 foundation; Figure 2 extends it for company workflows; Figure 3 adds private operation. The memory LLD applies across all editions. Version scope wording is preserved from the documented plan.

ORG - Company & Workspace

Architecture owner: Identity + Policy. **Proposed records:** Tenant, membership, invitation, role policy and workspace preference.

LLD rule: Membership changes invalidate authorization caches and queued delegated actions.

Feature / first edition	Scope and outcome	Architecture owner
ORG-E1 - Role-aware onboarding (2.0; Enhance)	Signup/login/logout, profile/settings, personal or development-team workspace, builder/developer/QA/designer entry views, invitations and all theme modes.	Identity + Policy
ORG-E2 - Project-scoped permissions (2.0; Enhance)	Project/team membership, owner/admin/builder/reviewer/viewer scopes, environment permissions and basic billing authority.	Identity + Policy
ORG-E3 - Collaborative project access (2.0; Enhance)	Project/doc/design invitations, comments, review access, ownership and safe removal.	Identity + Policy
ORG-E4 - Workspace budgets and administration (2.0; Enhance)	Visible credits, usage, estimates, plans/purchases, invoices, project budgets, payer visibility and BYOK distinction.	Identity + Policy
ORG-N1 - Role home and assigned work (2.0; New)	My projects, build/developer/QA queues, recent work and resume/deep-link behavior.	Identity + Policy
ORG-N2 - Company portfolio (3.0; New)	Group projects into products and programs; show owners, dependencies, milestones and delivery health without turning every user into an administrator.	Identity + Policy
ORG-N3 - Approval inbox (2.0; New)	Project-scoped plan/design/PR/test/release/access/budget decisions.	Identity + Policy
ORG-N4 - Reusable company workspace blueprints (3.0; New)	Versioned bundles of approved design systems, workflows, agent teams, model policies and connection requirements; preview dependencies and instantiate an independent project with approved local overrides.	Identity + Policy

PLAN - Product Planning & Docs

Architecture owner: Project context. **Proposed records:** Revisioned requirements, PRD/TRD, HLD/LLD, decision and planning artifacts.

LLD rule: Approve exact revisions and mark dependent artifacts stale when requirements change.

Feature / first edition	Scope and outcome	Architecture owner
PLAN-E1 - Living specification (2.0; Enhance)	Prompt-generated PRD/TRD, editable assumptions/acceptance criteria, versions and direct builder handoff; manual authoring optional.	Project context
PLAN-E2 - Discovery and prioritization (2.0; Enhance)	App idea clarification, simple source-backed scope and feasibility for the current project.	Project context
PLAN-E3 - Reusable planning templates (2.0; Enhance)	Project brief, PRD/TRD and technical planning templates.	Project context
PLAN-E4 - Collaborative artifact review (2.0; Enhance)	Project documents/design reviews, comments, version comparisons and implementation handoff.	Project context
PLAN-N1 - Sprint and Kanban delivery (3.0; New)	Epics, stories, tasks, bugs, backlog, sprint goals, capacity, estimates, owners, dependencies, blockers and retrospectives. AI proposes work; people approve scope and responsibility.	Project context
PLAN-N2 - Project wiki and documentation (2.0; New)	Project wiki, architecture decisions, API docs, setup guides and runbooks.	Project context
PLAN-N3 - Requirement-to-release traceability (2.0; New)	Requirement → source change → test → release traceability for apps.	Project context
PLAN-N4 - Collaborative brainstorming and decisions (3.0; New)	Shared idea board, alternatives, clustering, comments, voting, decision rationale and conversion to an opportunity/prototype/task.	Project context
PLAN-N5 - Strategic roadmap and trade-off planning (3.0; New)	Goals to initiatives/milestones, alternatives, capacity/dependencies, forecast assumptions and change impact across teams.	Project context

BUILD - App & Visual Builder

Architecture owner: Builder UI + Sandbox. **Proposed records:** Prompt session, preview, design-system reference, changeset and build activity.

LLD rule: Generated code stays isolated; user acceptance writes a new artifact revision, never an implicit release.

Feature / first edition	Scope and outcome	Architecture owner
BUILD-E1 - Guided and advanced build views (2.0; Enhance)	Complete prompt-first app builder for non-technical users plus code/terminal/advanced views for developers.	Builder UI + Sandbox
BUILD-E2 - Point-and-prompt editing (2.0; Enhance)	Select UI element/region → prompt → compare/apply/undo; persistent main composer.	Builder UI + Sandbox
BUILD-E3 - Design-system continuity (2.0; Enhance)	App design tokens/themes/responsive states; three platform themes each with light/dark/system; project design-system use.	Builder UI + Sandbox
BUILD-E4 - Understandable generation and recovery (2.0; Enhance)	Lead plus frontend/backend/data/integration builder specialists; visible generation, stop/queue/retry/checkpoints; independent testing.	Builder UI + Sandbox

Feature / first edition	Scope and outcome	Architecture owner
BUILD-N1 - Interactive journey map (2.0; New)	Connect screens, navigation, roles and loading/empty/error states into a map; select a journey to preview or send to QA.	Builder UI + Sandbox
BUILD-N2 - Approved component catalog (2.0; New)	Use and version approved components inside development projects.	Builder UI + Sandbox
BUILD-N3 - Optional waiting activities (2.0; New)	Optional multiple solo games/personal challenges while builds run; preserve all build controls.	Builder UI + Sandbox

CODE - Code, Repositories & Services

Architecture owner: Repository service + Sandbox. **Proposed records:** Repository binding, commit manifest, file tree, changeset and PR receipt.

LLD rule: Provider access is authoritative; pinned SHA and expected revision prevent overwriting newer work.

Feature / first edition	Scope and outcome	Architecture owner
CODE-E1 - Reliable repository onboarding (2.0; Enhance)	Enhance GitHub import with repository/branch selection, framework/runtime detection, setup diagnostics and ownership mapping. Show inaccessible repositories and missing permissions clearly.	Repository service + Sandbox
CODE-E2 - Controlled Git synchronization (2.0; Enhance)	Enhance outgoing commits with explicit branch context, sync status and conflict resolution. Provider state is authoritative; do not overwrite newer external changes.	Repository service + Sandbox
CODE-E3 - Reviewable change history (2.0; Enhance)	Diffs/history and compatible source/document/agent/config checkpoints; restore exclusions explicit.	Repository service + Sandbox
CODE-E4 - Runtime configuration (2.0; Enhance)	Environment configuration and scoped secret references for generated apps and managed execution.	Repository service + Sandbox
CODE-N1 - Browser development workspace (2.0; New)	Editor, terminal, search, language diagnostics, dependency management, run/debug commands and logs inside Architect; developers can use normal code tools alongside prompts.	Repository service + Sandbox
CODE-N2 - Pull-request workspace (2.0; New)	Open or create a PR; converse on its diff; propose fixes, run checks, request reviewers, resolve comments and merge only when repository policies allow. Display external updates and stale approvals.	Repository service + Sandbox
CODE-N3 - Multi-repository service workspace (2.0; New)	Attach several repositories to one product, map service/API dependencies and exact commit versions, run selected PRs with mocks or live dependencies, and coordinate related PRs and release order.	Repository service + Sandbox
CODE-N4 - Validated application container export (2.0; New)	Export generated application Docker/container packages and validate portability.	Repository service + Sandbox

AGENT - Agents & Bot Teams

Architecture owner: Agent harness + Orchestrator. **Proposed records:** Versioned agent definition, graph, run, child run, tool invocation and scoped memory.

LLD rule: Delegation is permission-narrowing; every tool call has a budget, scope and traceable receipt.

Feature / first edition	Scope and outcome	Architecture owner
AGENT-E1 - Unified agent lifecycle (2.0; Enhance)	Create/configure/test/version app agents; show app-agent inventory, runs and available usage/traces inside Architect.	Agent harness + Orchestrator
AGENT-E2 - Bot hierarchy and delegation (2.0; Enhance)	Multi-agent app build/test orchestration plus agent hierarchy used inside generated applications.	Agent harness + Orchestrator
AGENT-E3 - Recurring and event-driven work (2.0; Enhance)	Retain existing agent schedules/triggers and project build/test automation.	Agent harness + Orchestrator
AGENT-E4 - Permissioned memory and tool use (2.0; Enhance)	Project/app-agent memory/tool permissions, approved developer MCP/plugins, scoped context and human decisions.	Agent harness + Orchestrator
AGENT-N1 - Bot-team workroom (3.0; New)	Persistent conversation with a lead bot and specialist activity feed; assign a goal, inspect delegation, intervene, cancel work and review evidence-backed deliverables.	Agent harness + Orchestrator
AGENT-N2 - Framework adapters (2.0; New)	Developer-facing framework author/import/export adapters with explicit capability validation.	Agent harness + Orchestrator
AGENT-N3 - Bot-team operations (3.0; New)	Reusable team blueprints, ownership transfer, run budgets, tool health, version pinning and team evaluations; connect bot work directly to sprint tasks and approvals.	Agent harness + Orchestrator
AGENT-N4 - Dynamic specialist creation and delegated execution (3.0; New)	A lead or execution agent can propose or create temporary specialists from approved templates at runtime, within tool/data/budget/concurrency/depth limits; preserve lineage, results and lifecycle cleanup.	Agent harness + Orchestrator

MODEL - Models & Routing

Architecture owner: Model gateway. **Proposed records:** Model catalog, capability profile, routing policy, evaluation and usage ledger.

LLD rule: Honor price/privacy constraints; record exact model and route; keep provider keys server-side.

Feature / first edition	Scope and outcome	Architecture owner
MODEL-E1 - Unified model selection (2.0; Enhance)	Builder and generated-app agent model selection.	Model gateway
MODEL-E2 - Managed and customer-funded providers (2.0; Enhance)	Managed models, BYOK/OpenRouter-style routing and funding distinction with per-run costs.	Model gateway

Feature / first edition	Scope and outcome	Architecture owner
MODEL-E3 - Task-aware smart routing (2.0; Enhance)	Task-aware model routing for builds/app agents with capability, latency, budget and data rules.	Model gateway
MODEL-E4 - Single-model and per-agent presets (2.0; Enhance)	Single-model/per-agent/default routing presets for development.	Model gateway
MODEL-N1 - Private/open-weight endpoint catalog (2.0; New)	Connect approved accessible open-weight/private provider endpoints as an inference consumer.	Model gateway
MODEL-N2 - Workload evaluation and routing experiments (2.0; New)	Run the same representative tasks across eligible models, compare correctness/cost/latency, approve a routing policy and canary it before broad use.	Model gateway
MODEL-N3 - Inference operations console (2.0; New)	Developer endpoint/connector health, usage and quotas for project models.	Model gateway

DATA - Data, Integrations & Context

Architecture owner: Context + Connector gateway. **Proposed records:** Connection, source revision, extraction job, ACL and context manifest.

LLD rule: Source-level permissions are rechecked at retrieval and action time; secrets never become context.

Feature / first edition	Scope and outcome	Architecture owner
DATA-E1 - Unified data setup (2.0; Enhance)	Extend automatic backend setup with a visible schema, sample data, validation and migration review; distinguish demo storage from persistent backend data.	Context + Connector gateway
DATA-E2 - Connection lifecycle (2.0; Enhance)	GitHub, app backend/data/model connections, developer MCP/plugins, scope/health/reconnect/revoke flows.	Context + Connector gateway
DATA-E3 - Grounded project context (2.0; Enhance)	Project docs/images/references, citations, freshness and bounded builder context.	Context + Connector gateway
DATA-E4 - Generated-app identity (2.0; Enhance)	Generated-app login, roles, invitations and test identities distinct from Architect identity.	Context + Connector gateway
DATA-N1 - Architect through ChatGPT (3.0; New)	Expose selected Architect project tools through an authenticated ChatGPT integration: inspect status, draft requirements, request builds and open the exact result in Architect. Enforce the same permissions and review policies.	Context + Connector gateway
DATA-N2 - Existing-work system synchronization (3.0; New)	Connect issue trackers, docs and team chat with explicit ownership of each record, external IDs, conflict rules and sync status; avoid creating competing sources of truth.	Context + Connector gateway
DATA-N3 - Portable context packages (2.0; New)	Portable code/project docs/context and supported agent configuration, with dependency manifest.	Context + Connector gateway

QA - Quality & Test Lab

Architecture owner: Test + Release. **Proposed records:** Acceptance case, test matrix, fixture manifest, evidence artifact and gate decision.

LLD rule: Only recorded runner evidence tied to an exact changeset can satisfy a quality gate.

Feature / first edition	Scope and outcome	Architecture owner
QA-E1 - Acceptance-driven testing (2.0; Enhance)	App acceptance criteria, technical QA and app-owner UAT.	Test + Release
QA-E2 - UI verification and repair loop (2.0; Enhance)	UI/browser/accessibility tests and builder repair loop with evidence.	Test + Release
QA-E3 - Agent behavior evaluations (2.0; Enhance)	Generated-app/build-agent behavior tests and model comparisons.	Test + Release
QA-E4 - QA review and reproducibility (2.0; Enhance)	Extend reports/artifacts with replayable steps, screenshots, logs, severity, flaky-test labeling and sign-off tied to exact code, agent and model versions.	Test + Release
QA-N1 - Service test matrix (2.0; New)	Independent specialist UI/function/API/integration/security/performance tests appropriate to app; isolated/subset/full-stack service matrix.	Test + Release
QA-N2 - Preview-environment test data (2.0; New)	Managed isolated previews, seeded test identities/data, expiry and cleanup.	Test + Release
QA-N3 - Quality gate policy (2.0; New)	Combine code scans, dependencies, tests, agent evals and required human reviews into a release decision. Explain blockers and any authorized exception rather than hiding failed checks.	Test + Release

RELEASE - Release & Environments

Architecture owner: Release controller + Private lifecycle. **Proposed records:** Environment, deployment plan, signed image, provider receipt, install and recovery record.

LLD rule: Reviewed artifact and approved action hash precede execution; reconcile unknown provider outcomes.

Feature / first edition	Scope and outcome	Architecture owner
RELEASE-E1 - Environment-aware deployment (2.0; Enhance)	Preview/staging/production deployment flow for generated apps.	Release controller + Private lifecycle
RELEASE-E2 - Ownership and hosting choices (2.0; Enhance)	Managed or customer-owned generated-app hosting; repo/hosting ownership and costs.	Release controller + Private lifecycle
RELEASE-E3 - Domain and runtime readiness (2.0; Enhance)	Generated-app domains/TLS/environment/secrets and runtime health.	Release controller + Private lifecycle

Feature / first edition	Scope and outcome	Architecture owner
RELEASE-E4 - Publishing and release communication (2.0; Enhance)	App publication/listing with visibility, owner-cost disclosure and release notes.	Release controller + Private lifecycle
RELEASE-N1 - Coordinated service releases (2.0; New)	Pin the repository commits and service versions in a release manifest; sequence compatible changes and block incompatible API/data migrations.	Release controller + Private lifecycle
RELEASE-N2 - Gradual rollout and rollback (2.0; New)	Application staged rollout/rollback with artifact/schema compatibility warnings.	Release controller + Private lifecycle
RELEASE-N3 - Release readiness room (2.0; New)	Developer release readiness: checks, reviewers, owner decision, runbooks.	Release controller + Private lifecycle
RELEASE-N4 - Product retirement and migration (2.0; New)	Application deprecation/migration and owned resource cleanup.	Release controller + Private lifecycle
RELEASE-N5 - Cloud connections and private execution profiles (4.0; New)	Scoped cloud identities, private networks, runtime pools, provisioning plans, quotas and connection lifecycle.	Release controller + Private lifecycle
RELEASE-N6 - Private Architect distribution and lifecycle (4.0; New)	Platform installation, dependency readiness, upgrade, backup/restore and offline feasibility.	Release controller + Private lifecycle

OPS - Operations & Improvement

Architecture owner: Audit + Usage + Telemetry. **Proposed records:** Trace, cost allocation, alert, incident, recommendation and outcome record.

LLD rule: Keep estimates separate from actuals; remedial writes require scope and approval.

Feature / first edition	Scope and outcome	Architecture owner
OPS-E1 - Unified application and agent telemetry (2.0; Enhance)	Application and app-agent logs/traces/errors/runtime health and Lyrz-visible telemetry through validated adapters.	Audit + Usage + Telemetry
OPS-E2 - Role-specific operational reports (2.0; Enhance)	Builder/developer/QA/DevOps operational reports.	Audit + Usage + Telemetry
OPS-E3 - Run and spend investigation (2.0; Enhance)	Run/project/agent/model/API costs, credit ledger and confirmed vs estimated provider charges.	Audit + Usage + Telemetry
OPS-E4 - Improvement backlog (2.0; Enhance)	App errors/test failures and performance findings become developer improvement tasks.	Audit + Usage + Telemetry
OPS-N1 - Alerts and incident response (2.0; New)	Application/agent alerts, incident evidence, owner routing and recovery tasks.	Audit + Usage + Telemetry

Feature / first edition	Scope and outcome	Architecture owner
OPS-N2 - Supervised remediation (3.0; New)	An operations bot diagnoses a fault, proposes a patch or rollback, runs checks and requests the required action approval; record retries and prevent duplicate external side effects.	Audit + Usage + Telemetry
OPS-N3 - Release outcome tracking (2.0; New)	App release adoption/reliability metrics against project intent.	Audit + Usage + Telemetry
OPS-N4 - Cloud cost allocation and efficiency recommendations (3.0; New)	Join connected cloud billing and utilization data; allocate EC2/S3 and other supported service costs to projects/teams/environments; surface evidence-backed efficiency opportunities and reviewed implementation tasks.	Audit + Usage + Telemetry

CRM - Customer & Revenue Workspace

Architecture owner: Company records + Connector gateway. **Proposed records:** Account, contact, lead, opportunity, activity and commercial handoff.

LLD rule: Respect account ownership and contact consent; deduplicate external records and gate outbound actions.

Feature / first edition	Scope and outcome	Architecture owner
CRM-N1 - Accounts and relationship records (3.0; New)	Shared accounts, contacts, ownership, relationship history and stable identities. Customer-success views reuse these records.	Company records + Connector gateway
CRM-N2 - Lead capture, qualification and routing (3.0; New)	Capture/import, deduplicate, qualify, assign and retain source/contact preferences.	Company records + Connector gateway
CRM-N3 - Opportunity and pipeline management (3.0; New)	Stages, opportunities, next steps, commitments and conversion status using explicit definitions.	Company records + Connector gateway
CRM-N4 - Sales activities and follow-ups (3.0; New)	Prepare follow-ups, maintain activity history, schedule authorized bot work and expose failed or pending actions.	Company records + Connector gateway
CRM-N5 - Proposals and commercial handoffs (3.0; New)	Proposal records and customer commitments link to finance review and customer-success onboarding; renewal work is owned by CX-N4.	Company records + Connector gateway

CX - Customer Feedback & Success

Architecture owner: Company records + Project graph. **Proposed records:** Feedback, support case, onboarding milestone, health evidence and linked requirement.

LLD rule: Redact customer data in engineering handoffs; record consent and close-loop communication approval.

Feature / first edition	Scope and outcome	Architecture owner
CX-N1 - Feedback intake and customer evidence (3.0; New)	Capture source-linked feedback, group themes/duplicates and connect accounts to product opportunities.	Company records + Project graph

Feature / first edition	Scope and outcome	Architecture owner
CX-N2 - Support cases and escalations (3.0; New)	Case ownership, routing, escalation and links to technical incidents or product work.	Company records + Project graph
CX-N3 - Customer onboarding and activation (3.0; New)	Onboarding milestones, adoption tasks and customer-success handoff; metric definitions use shared BI capabilities.	Company records + Project graph
CX-N4 - Account health, retention and renewals (3.0; New)	Explainable health signals, renewal coordination, churn reasons and retention actions using shared account/commercial records.	Company records + Project graph
CX-N5 - Close the customer feedback loop (3.0; New)	Connect requests to decisions and shipped changes, prepare reviewed customer updates and record follow-up outcomes.	Company records + Project graph

FIN - Finance & Commercial Operations

Architecture owner: Company records + Approval inbox. **Proposed records:** Budget, purchase request, invoice exception, decision and forecast.

LLD rule: Finance scopes are restrictive; AI drafts are not payment authorization or reconciled financial truth.

Feature / first edition	Scope and outcome	Architecture owner
FIN-N1 - Business budgets and allocations (3.0; New)	Project/department budgets and approved investment limits; uses company approval/budget infrastructure.	Company records + Approval inbox
FIN-N2 - Purchase requests and procurement review (3.0; New)	Request, owner, supporting evidence, commercial review handoff and approval status.	Company records + Approval inbox
FIN-N3 - Invoice status and reconciliation exceptions (3.0; New)	Authorized invoice data, period/currency handling, matching exceptions and owner resolution.	Company records + Approval inbox
FIN-N4 - Commercial and investment decisions (3.0; New)	Review proposal terms, business cases, assumptions and funding decisions; CRM owns the originating proposal.	Company records + Approval inbox
FIN-N5 - Financial variance and forecasts (3.0; New)	Spend versus plan, explained variances and editable forecasts; shared BI/reporting supplies execution infrastructure.	Company records + Approval inbox

PEOPLE - People & Team Operations

Architecture owner: Company records + Identity policy. **Proposed records:** Directory, skills, access request, capacity and agreed goal.

LLD rule: Restrict people data; no autonomous employment ranking; offboarding revokes identities and delegation.

Feature / first edition	Scope and outcome	Architecture owner
PEOPLE-N1 - Employee directory and skills (3.0; New)	Authorized employee/team/skills records; ORG remains the authority for company membership and permissions.	Company records + Identity policy

Feature / first edition	Scope and outcome	Architecture owner
PEOPLE-N2 - Employee onboarding, offboarding and access requests (3.0; New)	Coordinate employee lifecycle tasks, access-request approvals and bot ownership transfer.	Company records + Identity policy
PEOPLE-N3 - Team capacity and workload (3.0; New)	Available capacity, workload, leave context and blockers; teams can correct incomplete context.	Company records + Identity policy
PEOPLE-N4 - Agreed goals and manager review context (3.0; New)	Role-aware goals, quality/outcomes evidence and human review, without treating activity counts as autonomous employment decisions.	Company records + Identity policy

BI - Business Intelligence & Goals

Architecture owner: Metrics catalog + Project graph. **Proposed records:** Metric definition, lineage, dataset query, dashboard, experiment and decision.

LLD rule: Use governed queries and semantic definitions; preserve freshness, lineage and uncertainty.

Feature / first edition	Scope and outcome	Architecture owner
BI-N1 - Metric catalog and definitions (3.0; New)	Owner, formula, source, timeframe, cohort, target and currency where relevant.	Metrics catalog + Project graph
BI-N2 - Instrumentation and data-quality planning (3.0; New)	Event/property plans, implementation links, duplicate/missing-event checks and data freshness.	Metrics catalog + Project graph
BI-N3 - Prompt-to-analysis and dashboards (3.0; New)	Authorized queries, charts, filters, segment comparisons and drill-down to supporting records; DATA owns connectors.	Metrics catalog + Project graph
BI-N4 - Business goals and decision tracking (3.0; New)	Company goals, targets and evidence-backed decision records linked to owned follow-up work.	Metrics catalog + Project graph
BI-N5 - Scheduled reporting and distribution (3.0; New)	Report audiences, cadence, snapshots/exports and delivery status; AGENT provides scheduling and permission infrastructure.	Metrics catalog + Project graph
BI-N6 - Experiments and outcome analysis (3.0; New)	Hypotheses, assignment/exposure, variants, measures, guardrails, data quality and explicit ship/revise/stop decisions.	Metrics catalog + Project graph

MKT - Marketing & Growth

Architecture owner: Company records + Connector gateway. **Proposed records:** Campaign, audience, asset revision, launch plan, publishing approval and attribution.

LLD rule: Publish only approved assets to approved recipients/channels; connector receipt establishes outcome.

Feature / first edition	Scope and outcome	Architecture owner
MKT-N1 - Campaign and audience planning (3.0; New)	Campaign briefs, audience definitions, calendars, tasks and accountable owners.	Company records + Connector gateway

Feature / first edition	Scope and outcome	Architecture owner
MKT-N2 - Content and asset review (3.0; New)	Drafts, approved claims, assets, revisions and content approval using shared document/review infrastructure.	Company records + Connector gateway
MKT-N3 - Go-to-market and launch coordination (3.0; New)	Positioning, packaging/pricing proposals, sales enablement, support readiness and dependency on actual release availability.	Company records + Connector gateway
MKT-N4 - Supported channel publishing (3.0; New)	Approved connected publishing actions, execution confirmation, drafts and failure/retry states; no assumption every channel is supported.	Company records + Connector gateway
MKT-N5 - Campaign performance and attribution (3.0; New)	Campaign outcomes, lead-source and attribution rules, conversion analysis and follow-up work using shared BI metric definitions.	Company records + Connector gateway

Appendix B: inherited interaction traceability

All 422 inherited interaction IDs are preserved below with their parent package. This is coverage of documented scope, not a claim that every interaction has a live backend. Detailed research evidence and treatment are in ARCHITECTURE.md.

Interaction ID	Feature	Parent package
ARC-01-01	Signed-in account and profile entry	ORG-E1
ARC-01-02	Organization selector	ORG-N2
ARC-01-03	My Account entry	ORG-E1
ARC-01-04	Credit balance	ORG-E4
ARC-01-05	Light/dark interface toggle	ORG-E1
ARC-01-06	Home navigation	ORG-N1
ARC-01-07	Agent Studio link	AGENT-E1
ARC-01-08	My projects	BUILD-E1
ARC-01-09	Published projects	BUILD-E1
ARC-01-10	Shared projects	ORG-E3
ARC-01-11	My Apps listing	BUILD-E1
ARC-01-12	All / Deployed / Shared with me filters	BUILD-E1
ARC-01-13	Newest-first sorting	BUILD-E1
ARC-01-14	App-name edit affordance	BUILD-E1
ARC-01-15	App identifier	BUILD-E1
ARC-02-01	Natural-language app brief	PLAN-E1
ARC-02-02	Prompt attachments / Add files	DATA-E3
ARC-02-03	Add existing Studio agents entry	AGENT-E1
ARC-02-04	Theme selection	BUILD-E3
ARC-02-05	Prompt library with business-use-case examples	PLAN-E3
ARC-02-06	What should I build / AI Consultant	PLAN-E2
ARC-02-07	Consultant name, email and optional organization form	PLAN-E2
ARC-02-08	Role-personalization entry	ORG-E1
ARC-02-09	How it works guidance	BUILD-E1

Interaction ID	Feature	Parent package
ARC-02-10	Marketplace / Agentlets entry	BUILD-E1
ARC-03-01	Theme Manager	BUILD-E3
ARC-03-02	58 presets visible in explored account	BUILD-E3
ARC-03-03	Automatic theme selection state	BUILD-E3
ARC-03-04	Preset search	BUILD-E3
ARC-03-05	Preset filters	BUILD-E3
ARC-03-06	Color/style grouping and tags	BUILD-E3
ARC-03-07	My themes tab	BUILD-E3
ARC-03-08	Organization themes tab	BUILD-E3
ARC-04-01	Import codebase	CODE-E1
ARC-04-02	Public GitHub URL or owner/repo entry	CODE-E1
ARC-04-03	Anonymous public-repository cloning described in dialog	CODE-E1
ARC-04-04	Default-branch behavior described in dialog	CODE-E1
ARC-04-05	Connect GitHub to browse repositories	CODE-E1
ARC-04-06	Private-repository import path	CODE-E1
ARC-05-01	Plan mode	PLAN-E1
ARC-05-02	Clarifying questions and suggested answers	PLAN-E1
ARC-05-03	User, outcome and persistence clarification	PLAN-E1
ARC-05-04	Default Lyzr agents option	AGENT-E1
ARC-05-05	GitAgent workbench option marked beta	CODE-E1
ARC-05-06	Product Requirements Document generation	PLAN-E1
ARC-05-07	Editable planning document	PLAN-E1
ARC-05-08	App mockup generation and review	PLAN-E4
ARC-05-09	Plan and mockup selection	PLAN-E4
ARC-05-10	Start building / Build with these handoff	BUILD-E1
ARC-05-11	Planning chat history	PLAN-E1
ARC-05-12	Stop generating control	BUILD-E4
ARC-06-01	Build mode	BUILD-E1

Interaction ID	Feature	Parent package
ARC-06-02	Chat-based change requests	BUILD-E1
ARC-06-03	Plan-mode toggle in composer	BUILD-E1
ARC-06-04	Build options and usage entry	OPS-E3
ARC-06-05	Streamed build progress	BUILD-E4
ARC-06-06	Named worker/delegation messages	AGENT-N1
ARC-06-07	Expandable action status groups	BUILD-E4
ARC-06-08	Step and token totals	OPS-E3
ARC-06-09	Commit identifier and change summary	CODE-E3
ARC-06-10	Revert-to-version control, disabled at captured state	CODE-E3
ARC-06-11	Optional feature testing toggle	QA-E1
ARC-06-12	Test this build entry	QA-E1
ARC-07-01	Plan / Agents / App workspace sections	BUILD-E1
ARC-07-02	App preview	BUILD-E1
ARC-07-03	Reload preview	BUILD-E1
ARC-07-04	Preview error state and recovery	BUILD-E1
ARC-07-05	Console entry	CODE-N1
ARC-07-06	Agent Management panel	AGENT-E1
ARC-07-07	Agent workflow visualization area / empty state	AGENT-E2
ARC-07-08	Edit Agents in Studio	AGENT-E1
ARC-08-01	View File Tree	CODE-N1
ARC-08-02	Expand/collapse source folders	CODE-N1
ARC-08-03	Refresh file tree	CODE-N1
ARC-08-04	File-selection viewer entry	CODE-N1
ARC-08-05	Download Code	CODE-E3
ARC-08-06	Environment variables marked beta	CODE-E4
ARC-08-07	Duplicate App	BUILD-E1
ARC-08-08	Clear Chat	BUILD-E1
ARC-08-09	Clear Agents	AGENT-E1

Interaction ID	Feature	Parent package
ARC-08-10	Delete App	BUILD-E1
ARC-09-01	Deploy dialog	RELEASE-E1
ARC-09-02	Analytics usage/performance entry	OPS-E2
ARC-09-03	Publish to Marketplace toggle	RELEASE-E4
ARC-09-04	App name	RELEASE-E4
ARC-09-05	Category selection	RELEASE-E4
ARC-09-06	Full description	RELEASE-E4
ARC-09-07	Generate description action	RELEASE-E4
ARC-09-08	Short description with 160-character limit	RELEASE-E4
ARC-09-09	Up to eight tags	RELEASE-E4
ARC-09-10	Custom-domain option on first deploy	RELEASE-E3
ARC-09-11	Lyzr-managed private GitHub repository option	RELEASE-E2
ARC-09-12	Own GitHub repository option	RELEASE-E2
ARC-09-13	Connect GitHub entry	DATA-E2
ARC-09-14	Deploy and publish action	RELEASE-E1
ARC-09-15	Notice that marketplace visits consume owner credits	ORG-E4
ARC-10-01	Total credits used	OPS-E3
ARC-10-02	App count	OPS-E3
ARC-10-03	My Usage / All Users	OPS-E3
ARC-10-04	Credit usage over time	OPS-E3
ARC-10-05	7-day / 30-day / 90-day / 12-month ranges	OPS-E3
ARC-10-06	Usage breakdown by app	OPS-E3
ARC-10-07	Refresh usage	OPS-E3
ARC-10-08	Empty usage state	OPS-E3
ARC-10-09	Promotional free-credit entry	ORG-E4
ARC-10-10	Resources	ORG-N1
ARC-10-11	Documentation	ORG-N1
ARC-10-12	Discord	ORG-N1

Interaction ID	Feature	Parent package
ARC-10-13	Lyzz University	ORG-N1
ARC-10-14	Help & Support	ORG-N1
ARC-10-15	Support chat	ORG-N1
ARC-10-16	Privacy, security and legal links	ORG-N1
ARC-11-01	Agent registry and Create Agent entry	AGENT-E1
ARC-11-02	Agent naming	AGENT-E1
ARC-11-03	Build / Playground / Deploy tabs	AGENT-E1
ARC-11-04	Role	AGENT-E1
ARC-11-05	Goal	AGENT-E1
ARC-11-06	Instructions editor	AGENT-E1
ARC-11-07	Expand instructions editor	AGENT-E1
ARC-11-08	Generate instructions	AGENT-E1
ARC-11-09	Improve instructions	AGENT-E1
ARC-11-10	Mention other agents with @	AGENT-E2
ARC-11-11	Managerial agent configuration	AGENT-E2
ARC-11-12	Attach native sub-agents	AGENT-E2
ARC-11-13	Attach A2A agents	AGENT-E2
ARC-11-14	Update agent	AGENT-E1
ARC-11-15	Version history	AGENT-E1
ARC-11-16	Agent settings	AGENT-E1
ARC-11-17	Git Agent entry, upgrade required	AGENT-E1
ARC-12-01	Provider/model picker	MODEL-E1
ARC-12-02	Suggest model	MODEL-N2
ARC-12-03	Compare models	MODEL-N2
ARC-12-04	Auto-route by complexity	MODEL-E3
ARC-12-05	Advanced model parameters	MODEL-E1
ARC-12-06	Temperature	MODEL-E1
ARC-12-07	Top P	MODEL-E1

Interaction ID	Feature	Parent package
ARC-12-08	Web-search context size control, disabled in captured state	MODEL-E1
ARC-12-09	Reasoning-effort control, disabled in captured state	AGENT-E1
ARC-12-10	Text examples	AGENT-E1
ARC-12-11	Structured JSON output/schema	AGENT-E1
ARC-12-12	Image output/provider selection	AGENT-E1
ARC-12-13	Downloadable file output: DOCX, PDF, CSV and PPT	AGENT-E1
ARC-13-01	Attach knowledge base	DATA-E3
ARC-13-02	Tool picker and tool search	DATA-E2
ARC-13-03	AI Suggest tool entry	DATA-E2
ARC-13-04	Composio tool entry	DATA-E2
ARC-13-05	Configure more tools	DATA-E2
ARC-13-06	Add skills	DATA-E2
ARC-13-07	Schedule	AGENT-E3
ARC-13-08	Trigger	AGENT-E3
ARC-13-09	Short- and long-term Memory	AGENT-E4
ARC-13-10	Data Query	DATA-E3
ARC-13-11	Context	DATA-E3
ARC-13-12	Voice	AGENT-E1
ARC-13-13	Responsible AI	AGENT-E4
ARC-13-14	Fairness/bias controls	AGENT-E4
ARC-13-15	Reflection	AGENT-E4
ARC-13-16	Groundedness	AGENT-E4
ARC-13-17	LLM-as-judge marked beta	QA-E3
ARC-14-01	Playground chat	QA-E3
ARC-14-02	Test sessions	QA-E3
ARC-14-03	Attachments	DATA-E3
ARC-14-04	Activity view	QA-E3
ARC-14-05	Artifacts view	QA-E3

Interaction ID	Feature	Parent package
ARC-14-06	API deployment instructions	AGENT-E1
ARC-14-07	Masked credential display	AGENT-E1
ARC-14-08	cURL request example	AGENT-E1
ARC-14-09	Agent JSON	AGENT-E1
ARC-14-10	A2A agent card	AGENT-E1
ARC-14-11	A2A discovery and execution instructions	AGENT-E1
ARC-14-12	App Store publish entry	AGENT-N3
ARC-15-01	SuperFlow list	AGENT-E2
ARC-15-02	New empty flow	AGENT-E2
ARC-15-03	Workflow templates	AGENT-E2
ARC-15-04	Import entry	AGENT-E2
ARC-15-05	Visual node canvas	AGENT-E2
ARC-15-06	Node search	AGENT-E2
ARC-15-07	Node settings	AGENT-E2
ARC-15-08	Generate node configuration with AI	AGENT-E2
ARC-15-09	Run as sub-agent control	AGENT-E2
ARC-15-10	Prompt editors	AGENT-E2
ARC-15-11	Model parameters and max tokens	MODEL-E1
ARC-15-12	Structured output fields and JSON editor	AGENT-E2
ARC-15-13	Undo / redo	AGENT-E2
ARC-15-14	Auto-align	AGENT-E2
ARC-15-15	Expand all nodes	AGENT-E2
ARC-15-16	Save	AGENT-E2
ARC-15-17	Execution history	AGENT-N1
ARC-15-18	Approvals	ORG-N3
ARC-15-19	Copy workflow as JSON	AGENT-E2
ARC-15-20	Run panel	AGENT-N1
ARC-15-21	Zoom in/out	AGENT-E2

Interaction ID	Feature	Parent package
ARC-15-22	Fit view	AGENT-E2
ARC-15-23	Minimap	AGENT-E2
ARC-15-24	Toggle canvas interactivity	AGENT-E2
ARC-16-01	AI Agent	AGENT-E2
ARC-16-02	A2A Agent	AGENT-E2
ARC-16-03	AI Swarm	AGENT-E2
ARC-16-04	Tool	AGENT-E2
ARC-16-05	LLM	AGENT-E2
ARC-16-06	Decide	AGENT-E2
ARC-16-07	Rank	AGENT-E2
ARC-16-08	Guardrails	AGENT-E2
ARC-16-09	Trigger	AGENT-E2
ARC-16-10	If	AGENT-E2
ARC-16-11	Switch	AGENT-E2
ARC-16-12	Merge	AGENT-E2
ARC-16-13	Filter	AGENT-E2
ARC-16-14	Loop	AGENT-E2
ARC-16-15	Wait	AGENT-E2
ARC-16-16	Wait for Approval	AGENT-E2
ARC-16-17	Stop & Error	AGENT-E2
ARC-16-18	No-Op	AGENT-E2
ARC-16-19	Set	AGENT-E2
ARC-16-20	Aggregate	AGENT-E2
ARC-16-21	Sort	AGENT-E2
ARC-16-22	Limit	AGENT-E2
ARC-16-23	Remove Duplicates	AGENT-E2
ARC-16-24	Rename Keys	AGENT-E2
ARC-16-25	TOON	AGENT-E2

Interaction ID	Feature	Parent package
ARC-16-26	Code	AGENT-E2
ARC-16-27	HTTP Request	AGENT-E2
ARC-16-28	Execute nested SuperFlow	AGENT-E2
ARC-16-29	Date/Time	AGENT-E2
ARC-16-30	Crypto	AGENT-E2
ARC-16-31	Parse Document	AGENT-E2
ARC-16-32	Extract Fields	AGENT-E2
ARC-16-33	Label Document	AGENT-E2
ARC-17-01	AI Email Reply with human approval	AGENT-N3
ARC-17-02	Ask the AI	AGENT-N3
ARC-17-03	Code Reviewer	AGENT-N3
ARC-17-04	Web Page Summarizer	AGENT-N3
ARC-17-05	Batch Sentiment Analyzer	AGENT-N3
ARC-17-06	Smart Email Triage	AGENT-N3
ARC-17-07	Research Swarm	AGENT-N3
ARC-17-08	ReAct Agent	AGENT-N3
ARC-18-01	Knowledge Base	DATA-E3
ARC-18-02	Knowledge Graph	DATA-E3
ARC-18-03	Semantic Model	DATA-E3
ARC-18-04	Skills	DATA-E2
ARC-18-05	Global Context	DATA-E3
ARC-18-06	Traces	OPS-E1
ARC-18-07	Transcripts	OPS-E1
ARC-18-08	Reports	OPS-E2
ARC-18-09	Models	MODEL-E1
ARC-18-10	Voice Providers	MODEL-E2
ARC-18-11	Tools	DATA-E2
ARC-18-12	Data Connectors	DATA-E2

Interaction ID	Feature	Parent package
ARC-18-13	Memory connections	AGENT-E4
ARC-18-14	Guardrails	QA-E3
ARC-18-15	Telephony	DATA-E2
ARC-18-16	Responsible AI	QA-E3
ARC-18-17	Simulation Engine	QA-E3
ARC-18-18	Improvement Engine	OPS-E4
ARC-18-19	Voice area	AGENT-E1
ARC-18-20	Blueprints	AGENT-N3
ARC-18-21	App Store	AGENT-N3
ARC-18-22	Global search	ORG-N1
ARC-18-23	Relaxed-view switch	ORG-N1
ARC-18-24	Help and resources	ORG-N1
ARC-18-25	Organization and credit controls	ORG-E4
ARC-19-01	Architect signup automatically provisions a Studio account; our session still encountered a separate Studio login	ORG-E1
ARC-20-01	PDF/DOCX processing for document-based retrieval	DATA-E3
ARC-20-02	CSV/Excel data analysis and visualization	BI-N3
ARC-20-03	Reuse Studio agents with their prompts, tools and knowledge bases	AGENT-E1
ARC-20-04	Apply theme fonts and styling to generated code	BUILD-E3
ARC-21-01	Role-specific pain-point interview	PLAN-E2
ARC-21-02	Existing-tool selection	DATA-E2
ARC-21-03	Additional business context	PLAN-E2
ARC-21-04	Three tailored agent recommendations	PLAN-E2
ARC-21-05	Capabilities and estimated time savings per recommendation	PLAN-E2
ARC-21-06	Generate PRD from a recommendation	PLAN-E1
ARC-22-01	Choose a preconfigured app	BUILD-E1
ARC-22-02	Provide organization/business context	BUILD-E1
ARC-22-03	Build My App provisions UI, agents and database	BUILD-E1

Interaction ID	Feature	Parent package
ARC-22-04	Example templates: customer service, vendor comparison, KPI dashboard, market sizing and LinkedIn content	BUILD-E1
ARC-23-01	Automatically provision app backend and database	DATA-E1
ARC-23-02	Generate data collections from app needs	DATA-E1
ARC-23-03	Database tab / visual collection explorer	DATA-E1
ARC-23-04	Live document viewer	DATA-E1
ARC-23-05	View schema and relationships	DATA-E1
ARC-23-06	Generate sign-up and sign-in screens	DATA-E4
ARC-23-07	Password hashing	DATA-E4
ARC-23-08	Login sessions and token handling	DATA-E4
ARC-23-09	Protect private routes	DATA-E4
ARC-24-01	QA agent runs generated code and attempts corrective changes	QA-E2
ARC-24-02	Generate agent orchestration, attach tools/knowledge and wire outputs into UI	AGENT-E1
ARC-24-03	Refine app and agent configuration through follow-up chat	BUILD-E1
ARC-24-04	App uses updated Studio agent behavior	AGENT-E1
ARC-25-01	Create owned GitHub repository and push initial code	CODE-E1
ARC-25-02	Automatically push subsequent changes as commits	CODE-E2
ARC-25-03	Managed hosting and public app URL	RELEASE-E1
ARC-25-04	Default app subdomain	RELEASE-E3
ARC-25-05	Usage and agent-trigger analytics	OPS-E2
ARC-25-06	Enterprise on-premise/private-VPC deployment	RELEASE-E2
ARC-26-01	Filter community apps by category	BUILD-E1
ARC-26-02	Filter by use case	BUILD-E1
ARC-26-03	Preview community apps	BUILD-E1
ARC-26-04	Inspect creator agent/prompt structure	BUILD-E1
ARC-27-01	Deploy and customize a Blueprint	AGENT-N3
ARC-27-02	Deploy App Store applications to workspace	AGENT-N3
ARC-27-03	Reviewed App Store submissions	AGENT-N3

Interaction ID	Feature	Parent package
ARC-27-04	Import external agents through A2A	AGENT-N2
ARC-27-05	Credential-backed A2A authentication	AGENT-N2
ARC-27-06	Trace and evaluate imported agents	QA-E3
ARC-27-07	Role-based team permissions	ORG-E2
ARC-27-08	Audit logs	ORG-E2
ARC-27-09	SSO/SAML	ORG-E2
ARC-28-01	Architect GitAgent workbench: beta option seen, not executed; UI describes home, journeys, wiki, skills and observe	CODE-E1
ARC-28-02	Architect environment variables: beta label	CODE-E4
ARC-28-03	Studio Git Agent: upgrade gate	AGENT-E1
ARC-28-04	Studio LLM-as-judge: beta label	QA-E3
ARC-28-05	Several Studio safety/advanced switches: unavailable on explored plan	QA-E3
ARC-28-06	SuperFlow GitAgent node: soon and disabled	AGENT-E2
ARC-28-07	SuperFlow Split Document node: soon and disabled	AGENT-E2
ARC-28-08	Marketplace Clone & Modify: coming soon in official docs; distinct from own-app Duplicate App	BUILD-E1
ARC-29-01	Preview a theme in light mode	BUILD-E3
ARC-29-02	Preview a theme in dark mode	BUILD-E3
ARC-29-03	Cards preview tab	BUILD-E3
ARC-29-04	Dashboard preview tab	BUILD-E3
ARC-29-05	Marketing preview tab	BUILD-E3
ARC-29-06	Palette preview tab	BUILD-E3
ARC-29-07	Apply Theme	BUILD-E3
ARC-29-08	Preview sample account, team, upgrade and cookie components; these are demonstrations of styling, not proof of platform billing/authentication controls	BUILD-E3
ARC-30-01	Search prompts	PLAN-E3
ARC-30-02	All Prompts category	PLAN-E3
ARC-30-03	Role/category tabs: General, Analysts, Marketing, Sales, Legal, HR, Support, Productivity, Development, Data & Analysis	PLAN-E3
ARC-30-04	Scroll categories left/right	PLAN-E3

Interaction ID	Feature	Parent package
ARC-30-05	Individual prompt titles and example briefs	PLAN-E3
ARC-31-01	Plan / App Mockup artifact tabs	PLAN-E4
ARC-31-02	Edit artifact	PLAN-E4
ARC-31-03	Copy artifact	PLAN-E4
ARC-31-04	Collapse artifact panel	PLAN-E4
ARC-31-05	Close artifact panel	PLAN-E4
ARC-31-06	View artifact from build setup	PLAN-E4
ARC-31-07	Close build setup	PLAN-E4
ARC-31-08	Suggested action to create an on-brand design file	BUILD-E3
ARC-31-09	Suggested action to draft a skill.md for the core agent	AGENT-E4
ARC-31-10	Learn more about GitAgent	CODE-E1
ARC-31-11	About plan mode help control	PLAN-E1
ARC-32-01	Message composer changes to a queue-message field during generation	BUILD-E4
ARC-32-02	Stop generation during iteration	BUILD-E4
ARC-32-03	Running task-status text	BUILD-E4
ARC-32-04	Failed-action status alongside successful actions	BUILD-E4
ARC-32-05	Disabled controls during generation	BUILD-E4
ARC-32-06	Expandable completed-step summary	BUILD-E4
ARC-32-07	Console count indicator	CODE-N1
ARC-32-08	Select control in preview toolbar, disabled in captured state; purpose not established	BUILD-E2
ARC-32-09	Dismissible toast	BUILD-E4
ARC-32-10	GitHub connection-checking state	DATA-E2
ARC-32-11	Empty project-list guidance	BUILD-E1
ARC-32-12	Disabled submit/deploy controls in incomplete forms	RELEASE-E1
ARC-32-13	Dialog cancel and close controls	BUILD-E4
ARC-33-01	Built with Architect attribution badge	BUILD-E1
ARC-33-02	Dismiss attribution affordance	BUILD-E1
ARC-33-03	Start building link on attribution	BUILD-E1

Interaction ID	Feature	Parent package
ARC-34-01	Login with Google	ORG-E1
ARC-34-02	Login with LinkedIn	ORG-E1
ARC-34-03	Login with GitHub	ORG-E1
ARC-34-04	Login with Email	ORG-E1
ARC-34-05	Sign up for Free entry	ORG-E1
ARC-34-06	Acquisition-source onboarding question	ORG-E1
ARC-34-07	Role-selection onboarding	ORG-E1
ARC-34-08	Onboarding Back control	ORG-E1
ARC-34-09	Skip to main content link	ORG-E1
ARC-34-10	Pin/unpin or open sidebar controls	ORG-E1
ARC-34-11	Global-search Command-K shortcut label	ORG-E1
ARC-34-12	Org / Me scope tabs	ORG-N1
ARC-34-13	Continue working / recent agents	ORG-N1
ARC-34-14	View all agents	AGENT-E1
ARC-34-15	View traces shortcut	OPS-E1
ARC-34-16	Home creation choices: Agent, Managerial, SuperFlow, Voice	AGENT-E1
ARC-34-17	Suggested blueprints: STORM Research, Blog Writing, Content Strategy, Agent Reach	AGENT-E1
ARC-34-18	Product-tour entry and Next/Close controls	ORG-E1
ARC-34-19	Monthly credit allowance, remaining amount/percentage and reset date	ORG-E4
ARC-35-01	New Chat	QA-E3
ARC-35-02	Collapse events panel	AGENT-N1
ARC-35-03	Resize editor panels	AGENT-E1
ARC-35-04	Feature search	AGENT-E1
ARC-35-05	Expand/collapse output-format panel	AGENT-E1
ARC-35-06	Enable model-parameter overrides	MODEL-E1
ARC-35-07	Save/cancel model parameters	MODEL-E1
ARC-35-08	Copy API request snippet	AGENT-E1

Interaction ID	Feature	Parent package
ARC-35-09	Show API key control with key masked by default	AGENT-E1
ARC-35-10	A2A Save card	AGENT-E1
ARC-35-11	A2A Add example	AGENT-E1
ARC-35-12	A2A Additional settings	AGENT-E1
ARC-35-13	How to use tab	AGENT-E1
ARC-36-01	Workflow list / approvals navigation	ORG-N3
ARC-36-02	Watch Demo entry	AGENT-E2
ARC-36-03	Name a new flow	AGENT-E2
ARC-36-04	Template groupings: Basics, Pipelines, Multi-Agent	AGENT-E2
ARC-36-05	Template capability badges and estimated-duration labels	AGENT-N3
ARC-36-06	Workflow breadcrumbs	AGENT-E2
ARC-36-07	Expand individual node	AGENT-E2
ARC-36-08	Reset node configuration	AGENT-E2
ARC-36-09	Add structured-output field	AGENT-E2
ARC-36-10	Delete Node	AGENT-E2
ARC-36-11	Hide runner	AGENT-N1
ARC-36-12	Close run panel	AGENT-N1
ARC-36-13	Input tab	AGENT-N1
ARC-36-14	Switch run input to JSON mode	AGENT-N1
ARC-37-01	Share icon on project card	ORG-E3
ARC-37-02	Invite collaborator by email	ORG-E3
ARC-37-03	Invite multiple people	ORG-E3
ARC-37-04	People with access list	ORG-E3
ARC-37-05	Recipient opens app from Shared with Me	ORG-E3
ARC-37-06	Collaborators edit the same app, rather than a fork; concurrent editing mechanics not verified	ORG-E3
ARC-38-01	Use tools for web search, email and external API actions	DATA-E2
ARC-38-02	Combine multiple service actions into a workflow	DATA-E2

Interaction ID	Feature	Parent package
ARC-38-03	Documented Dropbox connector: search/read/upload files, list/create/move folders, create file requests and Paper documents, account lookup and delete action	DATA-E2
ARC-38-04	Connector-specific authorization and action availability still require per-connector validation	DATA-E2
ARC-39-01	Organization Manage console	ORG-N2
ARC-39-02	Organization overview	ORG-N2
ARC-39-03	Isolated sub-accounts	ORG-N2
ARC-39-04	Team members, roles and usage	ORG-E4
ARC-39-05	Owner/Admin/Member roles	ORG-E2
ARC-39-06	Audit overview counts	ORG-E2
ARC-39-07	Audit action/resource breakdowns	ORG-E2
ARC-39-08	Audit log filters and refresh	ORG-E2
ARC-39-09	Event details	ORG-E2
ARC-39-10	Audit-log pagination	ORG-E2

Appendix C: cross-product handoff register

All 228 documented handoffs are listed. Each uses the ContextEnvelope, authorization recheck and recovery contract specified above. Opening a destination never counts as completing an external action.

ID	From	Action	To
H001	ORG-E1	Open my work	ORG-N1
H002	ORG-E1	Connect tools	DATA-E2
H003	ORG-E2	Track access request	ORG-N3
H004	ORG-E2	Return to collaboration	ORG-E3
H005	ORG-E3	Request a review	ORG-N3
H006	ORG-E3	Review the shared artifact	PLAN-E4
H007	ORG-E4	Investigate spend	OPS-E3
H008	ORG-E4	Review business allocation	FIN-N1
H009	ORG-N1	Open pending decisions	ORG-N3
H010	ORG-N1	Resume assigned work	PLAN-N1
H011	ORG-N2	Explore roadmap trade-offs	PLAN-N5
H012	ORG-N2	Inspect business goals	BI-N4
H013	ORG-N3	Resume delegated work	AGENT-N1
H014	ORG-N3	Continue release readiness	RELEASE-N3
H015	PLAN-E1	Review this specification	PLAN-E4
H016	PLAN-E1	Plan approved work	PLAN-N1
H017	PLAN-E2	Brainstorm alternatives	PLAN-N4
H018	PLAN-E2	Create a specification	PLAN-E1
H019	PLAN-E3	Complete the draft	PLAN-E1
H020	PLAN-E3	Publish reusable guidance	PLAN-N2
H021	PLAN-E4	Build the accepted design	BUILD-E1
H022	PLAN-E4	Create acceptance tests	QA-E1
H023	PLAN-N1	Implement the selected task	CODE-N1
H024	PLAN-N1	Delegate the selected task	AGENT-N1

ID	From	Action	To
H025	PLAN-N2	Open referenced implementation	CODE-N1
H026	PLAN-N2	Review the document	PLAN-E4
H027	PLAN-N3	Inspect verification evidence	QA-E4
H028	PLAN-N3	Inspect release outcomes	OPS-N3
H029	PLAN-N4	Validate the chosen idea	PLAN-E2
H030	PLAN-N4	Create a prototype	BUILD-E1
H031	PLAN-N5	Update delivery scope	PLAN-N1
H032	PLAN-N5	Return to the portfolio	ORG-N2
H033	BUILD-E1	Refine the preview	BUILD-E2
H034	BUILD-E1	Import existing code	CODE-E1
H035	BUILD-E2	Verify the visual change	QA-E2
H036	BUILD-E2	Inspect the resulting diff	CODE-E3
H037	BUILD-E3	Manage approved components	BUILD-N2
H038	BUILD-E3	Run design verification	QA-E2
H039	BUILD-E4	Review changes	CODE-E3
H040	BUILD-E4	Inspect run cost	OPS-E3
H041	BUILD-N1	Generate journey test cases	QA-E1
H042	BUILD-N1	Fix a selected screen	BUILD-E2
H043	BUILD-N2	Adjust design rules	BUILD-E3
H044	BUILD-N2	Review component changes	PLAN-E4
H045	BUILD-N3	Return to the running build	BUILD-E4
H046	CODE-E1	Open code workspace	CODE-N1
H047	CODE-E1	Add related services	CODE-N3
H048	CODE-E2	Open or create a PR	CODE-N2
H049	CODE-E2	Review history	CODE-E3
H050	CODE-E3	Review this change	CODE-N2
H051	CODE-E3	Inspect versioned test evidence	QA-E4
H052	CODE-E4	Check deployment readiness	RELEASE-E3

ID	From	Action	To
H053	CODE-E4	Resume runtime work	CODE-N1
H054	CODE-N1	Prepare review	CODE-N2
H055	CODE-N1	Test affected services	QA-N1
H056	CODE-N2	Inspect PR evidence	QA-E4
H057	CODE-N2	Prepare the reviewed release	RELEASE-N3
H058	CODE-N3	Run the service test matrix	QA-N1
H059	CODE-N3	Coordinate service releases	RELEASE-N1
H060	AGENT-E1	Evaluate this agent	QA-E3
H061	AGENT-E1	Use the agent in a workroom	AGENT-N1
H062	AGENT-E2	Start team work	AGENT-N1
H063	AGENT-E2	Save a reusable team	AGENT-N3
H064	AGENT-E3	Inspect a scheduled run	AGENT-N1
H065	AGENT-E3	Schedule a report	BI-N5
H066	AGENT-E4	Manage source context	DATA-E3
H067	AGENT-E4	Resolve access scope	ORG-E2
H068	AGENT-N1	Answer a bot decision	ORG-N3
H069	AGENT-N1	Track delegated delivery	PLAN-N1
H070	AGENT-N2	Choose compatible models	MODEL-E1
H071	AGENT-N2	Evaluate imported behavior	QA-E3
H072	AGENT-N3	Manage team schedules	AGENT-E3
H073	AGENT-N3	Review team spending	OPS-E3
H074	MODEL-E1	Connect a provider	MODEL-E2
H075	MODEL-E1	Compare task quality	MODEL-N2
H076	MODEL-E2	Resume model selection	MODEL-E1
H077	MODEL-E2	Monitor provider health	MODEL-N3
H078	MODEL-E3	Evaluate routing candidates	MODEL-N2
H079	MODEL-E3	Inspect fallback incidents	MODEL-N3
H080	MODEL-E4	Review specialist assignments	AGENT-E2

ID	From	Action	To
H081	MODEL-E4	Configure automatic routing	MODEL-E3
H082	MODEL-N1	Use the registered endpoint	MODEL-E1
H083	MODEL-N1	Monitor capacity	MODEL-N3
H084	MODEL-N2	Apply evaluated routing	MODEL-E3
H085	MODEL-N2	Reuse agent evaluation cases	QA-E3
H086	MODEL-N3	Investigate affected spending	OPS-E3
H087	MODEL-N3	Replace an endpoint	MODEL-N1
H088	DATA-E1	Set application identity	DATA-E4
H089	DATA-E1	Prepare isolated test data	QA-N2
H090	DATA-E2	Map synchronized records	DATA-N2
H091	DATA-E2	Attach allowed actions	AGENT-E4
H092	DATA-E3	Analyze the evidence	PLAN-E2
H093	DATA-E3	Set bot memory and context	AGENT-E4
H094	DATA-E4	Test app-user journeys	QA-E1
H095	DATA-E4	Validate auth configuration	RELEASE-E3
H096	DATA-N1	Continue the draft in Architect	PLAN-E1
H097	DATA-N1	Inspect requested bot work	AGENT-N1
H098	DATA-N2	Open synchronized tasks	PLAN-N1
H099	DATA-N2	Open synchronized accounts	CRM-N1
H100	DATA-N3	Continue documentation	PLAN-N2
H101	DATA-N3	Start work with selected context	AGENT-N1
H102	QA-E1	Review evidence	QA-E4
H103	QA-E1	Clarify acceptance criteria	PLAN-E1
H104	QA-E2	Refine the selected UI	BUILD-E2
H105	QA-E2	Review the repair	CODE-N2
H106	QA-E3	Revise the agent	AGENT-E1
H107	QA-E3	Compare model candidates	MODEL-N2
H108	QA-E4	Return findings to PR	CODE-N2

ID	From	Action	To
H109	QA-E4	Continue readiness review	RELEASE-N3
H110	QA-N1	Resolve service dependencies	CODE-N3
H111	QA-N1	Use verified service versions	RELEASE-N1
H112	QA-N2	Run acceptance cases	QA-E1
H113	QA-N2	Resolve setup variables	CODE-E4
H114	QA-N3	Review release eligibility	RELEASE-N3
H115	QA-N3	Resolve an exception decision	ORG-N3
H116	RELEASE-E1	Inspect deployed health	OPS-E1
H117	RELEASE-E1	Manage rollout or recovery	RELEASE-N2
H118	RELEASE-E2	Connect the owned repository	CODE-E1
H119	RELEASE-E2	Resume deployment	RELEASE-E1
H120	RELEASE-E3	Fix runtime configuration	CODE-E4
H121	RELEASE-E3	Deploy the ready target	RELEASE-E1
H122	RELEASE-E4	Coordinate launch readiness	MKT-N3
H123	RELEASE-E4	Prepare customer follow-up	CX-N5
H124	RELEASE-N1	Review coordinated readiness	RELEASE-N3
H125	RELEASE-N1	Inspect service health	OPS-E1
H126	RELEASE-N2	Inspect experiment outcome	BI-N6
H127	RELEASE-N2	Handle rollout incident	OPS-N1
H128	RELEASE-N3	Deploy approved candidate	RELEASE-E1
H129	RELEASE-N3	Obtain outstanding decisions	ORG-N3
H130	RELEASE-N4	Communicate migration status	CX-N5
H131	RELEASE-N4	Verify residual activity	OPS-E1
H132	OPS-E1	Open an incident	OPS-N1
H133	OPS-E1	Investigate referenced code	CODE-N1
H134	OPS-E2	Explore business context	BI-N3
H135	OPS-E2	Create an improvement	OPS-E4
H136	OPS-E3	Adjust eligible routing	MODEL-E3

ID	From	Action	To
H137	OPS-E3	Review budget variance	FIN-N5
H138	OPS-E4	Schedule improvement work	PLAN-N1
H139	OPS-E4	Validate the opportunity	PLAN-E2
H140	OPS-N1	Propose remediation	OPS-N2
H141	OPS-N1	Coordinate customer support	CX-N2
H142	OPS-N2	Inspect repair evidence	QA-E4
H143	OPS-N2	Execute controlled recovery	RELEASE-N2
H144	OPS-N3	Update goal decisions	BI-N4
H145	OPS-N3	Revise roadmap priorities	PLAN-N5
H146	CRM-N1	Open an opportunity	CRM-N3
H147	CRM-N1	Attach customer feedback	CX-N1
H148	CRM-N2	Create a qualified opportunity	CRM-N3
H149	CRM-N2	Prepare follow-up	CRM-N4
H150	CRM-N3	Prepare a proposal	CRM-N5
H151	CRM-N3	Request commercial review	FIN-N4
H152	CRM-N4	Schedule approved follow-up	AGENT-E3
H153	CRM-N4	Update opportunity context	CRM-N3
H154	CRM-N5	Review the terms	FIN-N4
H155	CRM-N5	Start customer onboarding	CX-N3
H156	CX-N1	Evaluate the opportunity	PLAN-E2
H157	CX-N1	Inspect affected accounts	CRM-N1
H158	CX-N2	Escalate an incident	OPS-N1
H159	CX-N2	Close the feedback loop	CX-N5
H160	CX-N3	Inspect activation definition	BI-N1
H161	CX-N3	Continue account health	CX-N4
H162	CX-N4	Prepare renewal terms	CRM-N5
H163	CX-N4	Investigate recurring product gaps	PLAN-E2
H164	CX-N5	Record customer communication	CRM-N4

ID	From	Action	To
H165	CX-N5	Review the delivered outcome	OPS-N3
H166	FIN-N1	Apply operational limits	ORG-E4
H167	FIN-N1	Monitor variance	FIN-N5
H168	FIN-N2	Review the request	ORG-N3
H169	FIN-N2	Track related invoice status	FIN-N3
H170	FIN-N3	Resolve source synchronization	DATA-N2
H171	FIN-N3	Update variance analysis	FIN-N5
H172	FIN-N4	Return reviewed proposal	CRM-N5
H173	FIN-N4	Update funded roadmap	PLAN-N5
H174	FIN-N5	Inspect source analysis	BI-N3
H175	FIN-N5	Revise authorized allocation	FIN-N1
H176	PEOPLE-N1	Inspect team capacity	PEOPLE-N3
H177	PEOPLE-N1	Manage actual access permissions	ORG-E2
H178	PEOPLE-N2	Complete access changes	ORG-E2
H179	PEOPLE-N2	Transfer bot ownership	AGENT-N3
H180	PEOPLE-N3	Adjust delivery assignments	PLAN-N1
H181	PEOPLE-N3	Review agreed goals	PEOPLE-N4
H182	PEOPLE-N4	Resolve workload blockers	PEOPLE-N3
H183	PEOPLE-N4	Link permitted team goals	BI-N4
H184	BI-N1	Implement missing instrumentation	BI-N2
H185	BI-N1	Build a report	BI-N3
H186	BI-N2	Implement tracking	CODE-N1
H187	BI-N2	Verify measurement criteria	QA-E1
H188	BI-N3	Record a decision	BI-N4
H189	BI-N3	Investigate a finding	PLAN-E2
H190	BI-N4	Adjust initiative priorities	PLAN-N5
H191	BI-N4	Review funding implications	FIN-N4
H192	BI-N5	Manage schedule	AGENT-E3

ID	From	Action	To
H193	BI-N5	Inspect the underlying report	BI-N3
H194	BI-N6	Apply rollout decision	RELEASE-N2
H195	BI-N6	Refine the hypothesis	PLAN-E2
H196	MKT-N1	Prepare campaign assets	MKT-N2
H197	MKT-N1	Review campaign allocation	FIN-N1
H198	MKT-N2	Check launch readiness	MKT-N3
H199	MKT-N2	Publish approved assets	MKT-N4
H200	MKT-N3	Confirm release readiness	RELEASE-N3
H201	MKT-N3	Execute channel plan	MKT-N4
H202	MKT-N4	Repair channel connection	DATA-E2
H203	MKT-N4	Measure campaign results	MKT-N5
H204	MKT-N5	Inspect lead quality	CRM-N2
H205	MKT-N5	Test a growth hypothesis	BI-N6
F-H01	ORG-N1	PM opens assigned requirement	PLAN-E1
F-H02	ORG-N1	Developer opens assigned PR	CODE-N2
F-H03	ORG-N1	QA opens test queue item	QA-E1
F-H04	ORG-N1	Designer opens assigned design	BUILD-E3
F-H05	PLAN-E1	Start coordinated build	BUILD-E4
F-H06	BUILD-E4	Verify build with independent specialists	QA-N1
F-H07	QA-E2	Repair UI finding	BUILD-E2
F-H08	BUILD-N3	Return from game to completed build	BUILD-E4
F-H09	BUILD-N2	Use approved design system	BUILD-E1
F-H10	BI-N3	Act on team metric	PLAN-N1
F-H11	AGENT-N1	Add missing specialist during run	AGENT-N4
F-H12	AGENT-N4	Collect specialist result	AGENT-N1
F-H13	AGENT-N1	Resolve contested or approval-gated decision	ORG-N3
F-H14	AGENT-E4	Attach MCP/plugin capability	DATA-E2
F-H15	OPS-E1	Inspect monitored agent run	AGENT-N1

ID	From	Action	To
F-H16	OPS-E3	Investigate model/API charge	MODEL-N3
F-H17	OPS-N4	Connect cloud cost visibility	RELEASE-N5
F-H18	OPS-N4	Review efficiency change	OPS-N2
F-H19	OPS-N4	Review resource budget variance	FIN-N1
F-H20	ORG-N4	Resolve blueprint connections	DATA-E2
F-H21	ORG-N4	Start from company blueprint	BUILD-E1
F-H22	DATA-N3	Export executable application package	CODE-N4
F-H23	CODE-E3	Revalidate restored checkpoint	QA-N3

Appendix D: shared experience and later utilities

HLD/LLD artifacts belong to planning and traceability. Themes, dark mode, sidebar navigation, profile/logout and the mascot are shared UI concerns. Waiting activities belong to BUILD-N3; bot-team delegation to AGENT; model/API/module costs to MODEL and OPS; infrastructure costs to OPS-N4. Home support is a later utility using the Model gateway and a curated help guide, not unrestricted company memory. The Markdown companion retains every edition-specific scope, actor, journey and recovery detail.